

Verity: Verifiable Local Differential Privacy

James Bell-Clark*
Google

Adrià Gascón†
Google

Baiyu Li‡
Google

Mariana Raykova§
Google

Amrita Roy Chowdhury¶
University of Michigan

May 12, 2025

Abstract

Local differential privacy (LDP) enables individuals to report sensitive data while preserving privacy. Unfortunately, LDP mechanisms are vulnerable to poisoning attacks, where adversaries controlling a fraction of the reporting users can significantly distort the aggregate output—much more so than in a non-private solution where the inputs are reported directly. In this paper, we present two novel solutions that prevent poisoning attacks under LDP while preserving its privacy guarantees. Our first solution, *Verity-Auth*, addresses scenarios where the users report inputs with a ground truth available to a third party. The second solution, *Verity*, tackles the more challenging case in which the users locally generate their input and there is no ground truth which can be used to bootstrap verifiable randomness generation.

1 Introduction

Collecting aggregate statistics while preserving individual privacy is of broad interest. Local differential privacy (LDP) provides a solution with strong privacy guarantees and minimal server requirements. Specifically, users perform all privacy-preserving computations locally, with the server only post-processing the responses to optimize utility. To achieve privacy, each user randomizes their input to introduce uncertainty about the individual values. However when aggregated across many users, the estimate still retains a weak signal that can be amplified and extracted. In other words, an LDP mechanism maintains utility by being highly sensitive to small changes in the distribution of the private inputs.

Unfortunately, the same property also leaves LDP mechanisms vulnerable to poisoning attacks, where malicious users submit false or malformed reports. In particular, poisoning the private reports can be far more harmful than poisoning the input itself. An adversary controlling a fraction of the users can disproportionately bias the server’s analysis, creating a larger distortion than if the adversary only tampered with the users’ inputs. Formally, under LDP, the adversary can amplify the impact of any poisoning of the input by a factor of $\frac{1}{\epsilon}$, where ϵ is the privacy parameter [CSU21].

In this work, we propose solutions that prevent poisoning attacks under LDP. In particular, we focus on randomized response (RR) [War65], one of the most fundamental and widely adopted LDP mechanisms, which is used for reporting binary inputs. A key advantage of RR is its non-interactive nature – each user prepares and submits their contribution without needing to maintain state or engage in further interactions. This is especially beneficial for devices with limited connectivity that may not be online for extended periods.

*jhbelle@google.com

†adriag@google.com

‡baiyuli@google.com

§marianar@google.com

¶aroyc@umich.edu

Additionally, it simplifies implementation and deployment. Therefore, a primary design goal for us is to preserve this valuable one-shot contribution feature of RR. We achieve this by enabling the users to generate a non-interactive cryptographic proof that their contribution results from a correct evaluation of RR.

Security Model. We consider adversaries who control a fraction of the users and can instruct them to tamper with their inputs or deviate from the RR mechanism, i.e., tamper with its randomness. Additionally, we allow malicious users to drop out and provide no response to the server, which can be modeled as submitting an invalid response (e.g., a non-binary value). Note that tampering or poisoning the input is always possible, even in the non-private case, whereas tampering with the randomizer is a characteristic of LDP. Hence, our security goal is to ensure that the output distribution matches that of an honest execution of RR directly on the users’ inputs. Specifically, our solutions restrict the adversary’s strategies to rely solely on the inputs—they cannot introduce additional bias by exploiting the randomness of RR.

Our Contributions

While a canonical application of RR involves reporting user-generated inputs, such as answers to survey questions, there are also scenarios where inputs have a ground truth available to a third party. For example, during the COVID-19 pandemic, exposure notification apps reported user infections across various demographics. In this case, the ground truth—whether a user tested positive—was available to the hospital conducting the tests. Another example is when a user reports some attributes about them in an online poll, such as, whether they are over 18, with the ground truth being a digital ID issued by the government. In the context of Privacy Sandbox [San23], an initiative to design privacy-preserving advertising APIs, RR is used to collect information about whether an ad led to a purchase – the ground truth for this bit is available to the advertiser. In these cases, the input to RR can be *authorized* by the third party. These bits can be reported on their own by the user to the server if it’s not possible to collect them from the authorizer. However, a more common scenario is when the bits are reported along with an associated payload, which is not known to the authorizer. For example, in an ad exchange, the payload contains information about the specific ad. Here, the *join* between the bit and the payload is known *only* to the user. We provide two novel constructions, *Verity-Auth* and *Verity*, for verifying RR for *both* input settings, i.e., with and without authorized inputs.

Verity-Auth: With Authorized Input. Verifiability in this setting involves two key tasks. First, the user must prove that they are using authorized inputs from the third party, hereafter referred to as the authorizer. Second, the user needs to demonstrate that the randomness used for RR was generated honestly. The high-level idea of *Verity-Auth* is as follows. User obtains a signature from the authorizer on the input. The user then generates the randomness for RR by using a pseudorandom function (PRF) [GGM84]. To ensure that the randomness is honestly generated while keeping it hidden from the server, the key for the PRF is derived jointly by both the user and the server. Finally, the user employs zero-knowledge proofs (ZKP) to demonstrate that their submitted report was generated by running RR on the signed (thereby, authorized) input, using the randomness generated by evaluating the PRF under the correct key. As mentioned above, we achieve this non-interactively.

Security. In addition to preventing tampering of randomizer, *Verity-Auth* can also prevent input tampering due to the access to authorized inputs. This means that in the case of no dropouts, *Verity-Auth* completely prevents poisoning attacks—i.e., the output distribution is exactly the same as if RR were executed honestly on the authorized inputs. While dropping out does give the adversary an opportunity to launch attacks, the effectiveness of such an attack now depends on the randomness of RR and the sum of the authorized bits – both of which are beyond the adversary’s control.

Efficiency. *Verity-Auth* is very efficient. Specifically, to authorize an input, a user takes 4.67ms while the authorizer runs in 2.05ms. To generate a verifiable RR bit from 3 pseudorandom bits, it takes the user 8.37ms to generate the required ZKP, where the proof is only 1.19KB; and it takes the server 3.54ms to

verify both ZKP and signature. Compared to a concurrent work [BAT24], that operates in a weaker threat model, *Verity-Auth* is 100× more efficient.

Verity: Without Authorized Input. The scenario in which the user generates their own input is more challenging because merely ensuring the honest generation of randomness is not enough—a malicious user could still carry out arbitrary poisoning attacks by changing the input based on the sampled randomness (see Sec. 5). *Verity* addresses this by keeping the randomness encrypted from the user. To preserve non-interactivity, we enable the user to evaluate the PRF under homomorphic encryption (HE). However, we must also enforce verifiability on top of this homomorphic computation. Achieving this through generic composition with ZKPs is extremely inefficient. Instead, *Verity* adopt a different approach by leveraging the deterministic nature of HE ciphertexts. Specifically, the server can verify the computation by rerunning the homomorphic operation on the encrypted inputs and checking that it obtains identical ciphertexts. To the best of our knowledge, this is first work to provide robustness by keeping the randomness encrypted.

Security. *Verity* ensures the adversary cannot introduce more bias than by tampering with the inputs alone.

Efficiency. The user takes 1424s to homomorphically generate a verifiable RR bit from 3 pseudorandom bits, while the server verifies in 1473s. Using the alternative ZKP-based approach for our functionality is currently practically infeasible (see Sec. 9.2).

2 Background

2.1 Local Differential Privacy

We focus on the *local* model of DP which consists of a set of individual users (U) and an untrusted server.

Definition 1 (LDP [DR14]). *A randomized mechanism $\mathcal{M} : \mathcal{X} \mapsto \mathcal{Y}$ satisfies ϵ -LDP if for any $x, x' \in \mathcal{X}$ and any $y \in \mathcal{Y}$, we have*

$$\Pr[\mathcal{M}(x) = y] \leq e^\epsilon \Pr[\mathcal{M}(x') = y]$$

Randomized Response. Randomized response ($RR_\rho : \{0, 1\} \times [0, 1] \mapsto \{0, 1\}$) is a classic LDP mechanism which releases a single bit $b \in \{0, 1\}$ by flipping it with probability $\rho = \frac{1}{1+e^\epsilon}$.

Theorem 1 ([DR14]). *Randomized response, RR_ρ satisfies ϵ -LDP.*

2.2 Cryptographic Primitives

Legendre PRF. We use the Legendre PRF [Dam90], denoted as $\mathcal{F} : \mathbb{F}_p \times \{0, 1\}^\kappa \mapsto \{0, 1\}$, where p is a κ -bit prime. This is a one-bit PRF defined using the Legendre symbol ($\frac{a}{p}$) as follows:

$$\mathcal{F}(x, K) = \left\lceil \frac{1}{2} \left(\left(\frac{K+x}{p} \right) + 1 \right) \right\rceil \quad (1)$$

The security of $\mathcal{F}$ as a PRF has been studied in [GRR⁺16, SHB23, BBUV20, KKK20b]. Let n be a quadratic nonresidue in $\mathbb{F}_p$. We can verify the evaluation of the Legendre PRF, i.e., prove $\mathcal{F}(x, K) = b$ as follows:

- Prove that b is binary, i.e., $b(1-b) = 0$.
- If $b = 0$, set $w = \sqrt{n(K+x)}$; otherwise set $w = \sqrt{K+x}$.
- Prove that $w^2 = ((1-b)n+b)(K+x)$ with w as the witness.

Non-Interactive Zero Knowledge Argument of Knowledge. We use Schnorr’s protocols [Ode09a] and Bulletproofs [BBB⁺18] under Fiat-Shamir transformation.

Pedersen Commitment. We use the Pedersen commitment over a cyclic group $\mathbb{G}$ of order p . Let P, Q be two distinct group generators. For $x \in \mathbb{Z}_p$, a Pedersen commitment of x is $\text{com}_x = xP + rQ$, where $r \leftarrow \mathbb{Z}_p$, and its opening is (x, r) .

EQS Blind Signature. We use the equivalence class based blind signatures (EQS) [FHS19], that can sign messages that are vectors of group elements $\vec{M} \in (\mathbb{G}_1^*)^l, l \in \mathbb{Z}_{>0}$:

- $(\text{sk}, \text{pk}) \leftarrow \text{EQS.GenParam}(1^\kappa)$: On input the security parameter κ , the signer outputs a private parameter sk and a public parameter pk .
- $(\vec{M}, \text{st}) \leftarrow \text{EQS.SignRequest}(x, \text{pk})$: On input $x \in \mathbb{Z}_p$ and a public parameter pk , the user outputs a signing request $\vec{M}$ for x and a local state st .
- $\sigma \leftarrow \text{EQS.Sign}(\text{sk}, \vec{M})$: On input a signing request $\vec{M}$ and a private parameter sk , the signer outputs a signature σ .
- $\sigma' \leftarrow \text{EQS.Unblind}(\text{st}, \sigma)$: The user unblinds σ and obtains a signature σ' for their input x .

The exact construction is in Fig. 7. Here, a signature for $\vec{M}$ can be adapted into signatures of any multiple $\mu\vec{M}, \mu \in \mathbb{Z}_p^*$. EQS signature satisfies the usual existential unforgeability under adaptive chosen-message attack (EU-CMA). For *blindness*, we consider the *malicious signer* (Def. 2).

Threshold Homomorphic Encryption. We use a (2,2) threshold leveled homomorphic encryption scheme **HE**, in which the master secret key is additively shared between two parties.

- $(\text{pk}, \text{sk}_1, \text{sk}_2) \leftarrow \text{HE.GenParam}(1^\kappa)$: On input the security parameter κ the algorithm **GenParam** outputs a public key pk and shares of the secret key $(\text{sk}_1, \text{sk}_2)$ for the two parties.
- $\text{ct} \leftarrow \text{HE.Enc}(\text{pk}, x)$: The encryption algorithm outputs a ciphertext ct encrypting the plaintext input x .
- $\text{ct}' \leftarrow \text{HE.Eval}(\text{pk}, C, (\text{ct}_i)_{i=1}^k)$: The evaluation algorithm takes a public key pk , a circuit C representing a k -ary function f , and a tuple of ciphertexts $(\text{ct}_i)_{i=1}^k$, where each ct_i encrypts a plaintext input x_i , and it outputs a ciphertext ct of $f(x_1, \dots, x_k)$.
- $d_i \leftarrow \text{HE.PartialDec}(\text{sk}_i, \text{ct})$: The partial decryption algorithm takes a secret key share sk_i and a ciphertext ct , and it outputs a partial decryption result d_i .
- $x' \leftarrow \text{HE.Combine}(d_0, d_1)$: The recovery algorithm combines partial decryption results d_0 and d_1 from two users, and it outputs a fully decrypted plaintext x' .

HE should satisfy IND-CPA security, and a partial decryption result d_i should hide the secret key share sk_i . We also require that the **HE** instantiation ensures **HE.Eval** is deterministic.

3 Problem Overview

3.1 Problem Setting

Every user $\mathcal{U}_i, i \in [n]$ holds a private bit $x_i \in \{0, 1\}$. The users report the corresponding noisy bits, $y_i = \text{RR}_\rho(x_i)$, to an untrusted server, $\mathcal{S}$, which estimates the aggregate as $\hat{s} = \frac{1}{1-2\rho}(\sum_{i=1}^n y_i - \rho n)^1$.

Additionally, we consider the option of a third-party authorizer $\mathcal{A}$ holds the ground truth of the bits x_i^A . As discussed in Sec. 1, in this case, the user holds additional information payload_i and reports $(y_i, \text{payload}_i)$ to the server. For example, in a COVID-19 setting, the payload might contain demographic data, while in an ad exchange, it could include information about a specific ad. The authorizer does not have access to the user's payload. In particular, the *join* of the private bit and the payload $(x_i, \text{payload}_i)$ is known *only* to the user. The server here computes a histogram of the bits, bucketed by the payloads. In the extreme case

¹ $\hat{s}$ is the unbiased estimate of the sum of the private bits, i.e., $\mathbb{E}[\hat{s}] = \sum_{i=1}^n x_i$.

Privacy	Authorized Input	Honest Randomness	ℓ_1 Error	Notes		
No privacy	✗	N/A	m	Error only due to input tampering; randomizer tampering is not applicable.		
	✓	N/A	0	Preventing input tampering completely eliminates poisoning attacks.		
ϵ -LDP	✗	✗	$\tilde{O}(m(1 + \frac{1}{\epsilon}) + \frac{\sqrt{n}}{\epsilon})$	Corresponds to the worst-case attack. Observe that privacy increases the strength of poisoning. In particular, $\frac{1}{\epsilon}$ factor comes from de-biasing while $\frac{\sqrt{n}}{\epsilon}$ is the inherent error due to the noise to satisfy LDP.		
	✓	✗	$\tilde{O}(m(1 + \frac{1}{\epsilon}) + \frac{\sqrt{n}}{\epsilon})$	Preventing just input tampering does not give any protection. A malicious user can still carry out the worst-case attack by tampering with the randomizer.		
	✗	✓	Clear	$\tilde{O}(m(1 + \frac{1}{\epsilon}) + \frac{\sqrt{n}}{\epsilon})$	Revealing the noise in the clear provides no protection as the user can lie about their input and still carry out the worst-case attack.	
			Encrypted	$\tilde{O}(m + \frac{\sqrt{n}}{\epsilon})$	This corresponds to <i>Verity</i> (Fig. 4). The user is restricted to only input tampering which results in a weaker attack.	
	W/O drop-outs	✓	✓	Clear	$\Theta(\frac{\sqrt{n}}{\epsilon})$	This corresponds to <i>Verity-Auth</i> (Fig. 3). Poisoning attack is completely prevented; the only error term $O(\frac{\sqrt{n}}{\epsilon})$ is inevitable due to noise.
			Encrypted	$\Theta(\frac{\sqrt{n}}{\epsilon})$	This corresponds to <i>Verity-Auth</i> ⁺ (Fig. 8). Encrypting the randomness is unnecessary and results in extra computational overhead.	
W/ drop-outs	✓	✓	Clear	$\tilde{O}(m(1 + \frac{1}{\epsilon}) + \frac{\sqrt{n-m}}{\epsilon})$ w. p. $\left(\frac{e^\epsilon}{1+e^\epsilon}\right)^q \left(\frac{1}{1+e^\epsilon}\right)^{m-q}$	Malicious users can drop-out <i>based on the sampled randomness</i> . Consequently, they can still carry out the worst-case attack but with a much lower probability.	
			Encrypted	$\tilde{O}(q + \frac{\sqrt{n}}{\epsilon})$	Drop-outs can be only on the basis of the inputs now which is inevitable.	

Table 1: n denotes the total number of users, m is the number of malicious users and q is the sum of the input bits of all malicious users. Authorized input means that input tampering is prevented while honest randomness implies randomizer tampering is prevented. For the latter, we consider two scenarios - one where the randomness (equivalently, sample of random noise) is revealed in the clear to the user and the another, where it is kept hidden via encryption. Drop-out attack means a malicious user sends no response/ invalid response to the server; drop-outs can be based on either the sampled randomness or just the input. The above results assume the single message setting and unless explicitly stated, hold with arbitrarily high probability.

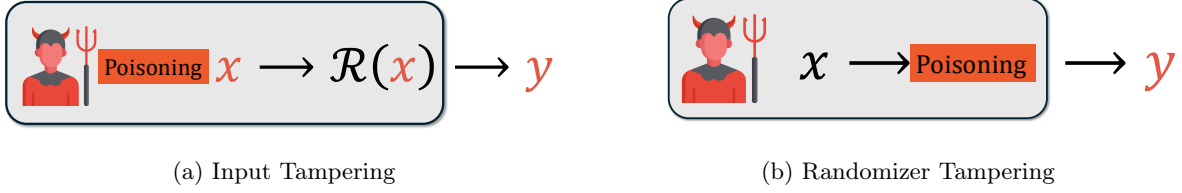


Figure 1: Poisoning attacks on an LDP protocol.

where all bits end up in the same bucket, this reduces to aggregating the noisy y_i s. Thus, the ℓ_1 error of the histogram is asymptotically bounded by that of the aggregate $\hat{s}$. For simplicity, we assume that the server computes the aggregate $\hat{s}$ instead of the histogram.

We do *not* consider either the privacy or the authenticity of the payload. In other words, the user is free to select any payload to report with their bit. Since our primary goal is verifying the correctness of the LDP mechanism, authenticating the non-private payload is currently beyond the scope of our work.

3.2 Security Model

We consider a *malicious adversary* threat model:

- **Malicious User.** We consider a set of m malicious user. A malicious user can deviate from the protocol in any way they chose and send a *poisoned* response. Additionally, the malicious users are allowed to collude together.
- **Malicious Server.** We consider a malicious server that may arbitrarily deviate from the protocol to recover the inputs x_i s.
- **Malicious Authorizer.** In the setting with authorized inputs, the authorizer may also act maliciously. Additionally, both the the server and the authorizer are allowed to collude together.

Impact of Poisoning. A typical LDP protocol (Fig. 1) processes a user’s private input x through a *randomizer*, which adds noise to produce a noisy output or response y . This separation between input x and response y creates two points of attack for an adversary:

- **Input Tampering.** First, the adversary can tamper with the input, which occurs when users cannot access the randomizer’s implementation—such as in mobile apps running proprietary code. In this case, a malicious user can only falsify their input from x_i to an arbitrary x'_i , and report $y_i = \text{RR}_\rho(x'_i)$.
- **Randomizer Tampering.** The second opportunity occurs when the adversary controls the randomizer’s implementation, such as by hacking the mobile app collecting data. In this case, a malicious user can submit an arbitrary response y_i instead of generating the correct noise to randomize their input.

Input tampering is possible for *any* protocol, private or not, because a user is free to change their input anytime. However, randomizer tampering attacks are unique to LDP – the distinction between a user’s input and their response is a characteristic of LDP.

Randomizer tampering enables a strong poisoning attack as follows:

Lemma 2 (Randomizer Tampering [CSU21]). *Let us assume m malicious users. They can introduce an ℓ_1 error of $\tilde{O}(m(1 + \frac{1}{\epsilon}) + \frac{\sqrt{n}}{\epsilon})$ in the server’s estimate, $\hat{s}$, with arbitrarily small probability of failure.*

Intuitively, the $\frac{\sqrt{n}}{\epsilon}$ term is due to the error from RR_ρ . The $m(1 + \frac{1}{\epsilon})$ term comes from the adversarial behavior of the malicious users – m term is inevitable and accounts for the worst case scenario where all m malicious users are colluding and perturbs their responses in the same direction (i.e., say reports a 1), while the $\frac{1}{\epsilon}$ factor corresponds to the scaling factor required for de-biasing.

If the malicious users are restricted to input tampering only, the efficacy of the attack weakens:

Lemma 3 (Input Tampering [CSU21]). *If m users execute an input poisoning attack, they can skew the server’s estimate $\hat{s}$ by an ℓ_1 error of $\tilde{O}(m + \frac{\sqrt{n}}{\epsilon})$ with arbitrarily high probability.*

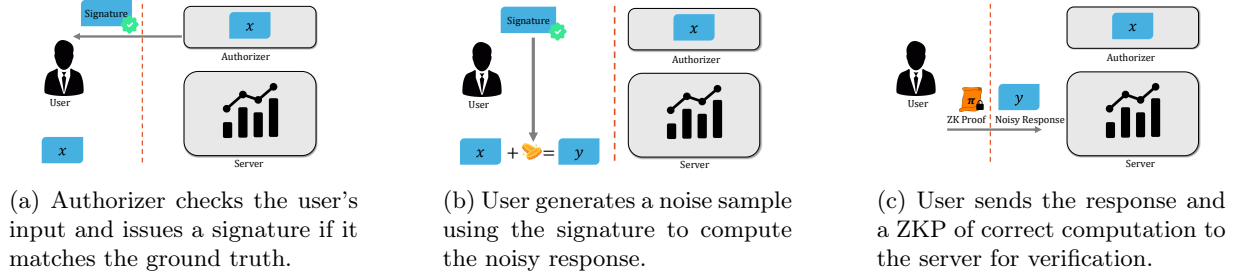


Figure 2: High-level overview of *Verity-Auth*.

Compared to Lemma 2, the input tampering attack is weaker by a factor of $\frac{m}{\epsilon}$. This demonstrates that vanilla LDP makes the protocol more vulnerable to poisoning attacks, as the adversary can introduce additional error by exploiting the randomness.

Drop Outs. In addition to the aforementioned attacks, malicious users can drop out, which can be modeled as submitting an invalid response (e.g., a non-binary value). Ensuring that users always provide valid responses requires incentives or enforcement mechanisms, which are application-specific and are orthogonal to our goals. We analyze the robustness of our solutions in both settings – when the adversary can cause users to drop out and when users are mandated to submit a valid response.

Drop outs are interesting only in the setting with verifiability. In the absence of verifiability, drop outs do not grant the adversary any advantage since they can fully control the value that each malicious user submits. However, when a user's response can be verified, the adversary may decide to drop out based on the randomness of RR (sampled noise). Hence, we aim to restrict the adversary to making the decision to drop out based solely on the input and *not* on the randomness sampled in RR. To the best of our knowledge, this is first paper to explicitly analyze drop out attacks for LDP.

Table 1 summarizes the robustness guarantees of our solutions.

4 *Verity-Auth*: With Authorized Input

Here, we describe our solution, *Verity-Auth*, that provides provable robustness against poisoning attacks when users have authorized inputs. In particular, *Verity-Auth* prevents randomizer tampering by cryptographically enforcing the correct evaluation of RR. Additionally, *Verity-Auth* prevents input tampering by having the authorizer perform a consistency check on the user's input against the ground truth, thereby ensuring its validity.

4.1 Technical Intuition.

Verity-Auth works in three phases (Fig. 2). In the first phase², the authorizer, who has the ground truth of the input, performs a consistency check on the user's input bit. In case the consistency check passes, the authorizer issues a signature and sends it to the user. In the second phase, the user uses this signature to sample an instance of random noise, which is then used to generate the noisy bit. However, since the user could be malicious, we cannot trust them to have followed the steps so far correctly. Hence, in the last phase, the server needs to verify the correctness of the user's computation. To accomplish this, the user sends a cryptographic proof alongside the noisy bit to the server. The server then accepts the user's response only if the proof is valid. However, designing this three-phase protocol is not straightforward. In particular, our solution must satisfy the following desiderata.

- *Unlinkability.* First, the input consistency check must ensure unlinkability – the user's interaction with the authorizer in the first round and their interaction with the server in the final round should not

²Note that obtaining an authorized input from $\mathcal{A}$ is inherent to our problem setting here and, therefore, is not considered a violation of the desired one-shot interaction.

be traceable to a *single* user through the messages communicated. More precisely, the user’s report $(y_i, \text{payload}_i)$ to the server should be not linked with the authorizer’s ground truth x_i^A . Otherwise they could collude to recover the exact join $(x_i, \text{payload}_i)$, which is a violation of the user’s privacy.

- *Non-Interactivity.* The second phase of the protocol involves generating a sample of noise securely. While previous work has addressed this issue [KMS⁺23, CasU19, BC23, DKM⁺06, KCY21, MN06, AJL03], all existing solutions require interaction. However, as motivated in Sec. 1, we aim to securely sample noise non-interactively.
- *Efficiency.* The final task of verifying the user’s computation is done with the help of ZKPs. However, ZKPs are typically extremely computationally intensive, so the challenge is to design an efficient solution that performs well in practice.

4.2 Workflow

The protocol can be broken into three phases with an initial one-time setup phase. The full protocol is outlined in Fig. 3.

Setup Phase. The setup phase is used to initialize the parameters for various cryptographic primitives. Specifically, the authorizer $\mathcal{A}$ generates the parameters for the EQS based blind signature scheme:

$$(\text{pprm}_{\mathcal{A}}, \text{privprm}_{\mathcal{A}}) \leftarrow \text{EQS.GenParam}(1^\kappa).$$

Additionally, each user $\mathcal{U}_i$ samples a secret key $\text{sk}_i \leftarrow \mathbb{F}_p$ for the Legendre PRF and sends a commitment to it, $\text{com}(\text{sk}_i)$, to the server. sk_i will be used to sample noise, which we will discuss later.

Input Authorization Phase. In a nutshell, the input consistency check is performed using a *blind signature* issued by the authorizer $\mathcal{A}$. Let x_i^A be the ground truth of $\mathcal{U}_i$ ’s input bit, which is known to the authorizer $\mathcal{A}$. The user $\mathcal{U}_i$ submits a blinded message containing a commitment com_{x_i} to their input bit x_i , along with a ZK proof of the opening of com_{x_i} . If the user acts truthfully, com_{x_i} should commit to the ground truth bit x_i^A – the authorizer validates the proof to ensure this consistency. If the proof is valid, the authorizer $\mathcal{A}$ issues a signature σ_i on the blinded message. The user then unblinds this signature to construct a new, valid signature σ'_i , which is used later in the verification phase with the server. Specifically, σ'_i is obtained by *re-randomizing* σ_i which ensures the unlinkability of the user’s interactions with the authorizer and the server. In what follows, we elaborate on the details of this construction.

As depicted in Fig. 2, the signature σ_i obtained from the authorizer is used in both of the two phases following the input authorization:

- A token τ_i is derived from the signature σ_i and is used for sampling noise.
- The re-randomized signature σ'_i is used in the verification phase to prove that the user performed their computation (of running RR) on an authorized input.

Additionally, it is important to *bind* the user’s payload and secret key to their input x_i , *prior* to sampling noise. Recall that our overarching security goal is to prevent the adversary from exploiting the sampled noise of RR in any way. Hence, binding the payload beforehand prevents a malicious user from choosing the payload *after* observing the sampled noise. The rationale for binding the secret key is more involved. The high-level intuition is that, without such a binding, malicious users could swap inputs based on the token τ_i and carry out an attack. We explain this with an example in App. B.

This necessitates the ability to issue a blind signature on a vector of values, where the input bit, token τ_i , and the fingerprints of sk_i and payload_i are all components of the vector. We accomplish this by using an EQS-based blind signature with some modifications, which we describe below. Recall, that in the standard EQS scheme, the message is a vector of length two given by $\vec{M}_i = (h\text{com}_{x_i}, hP)$ where $\text{com}_{x_i} = x_iP + rQ$, with

$r \leftarrow \mathbb{Z}_p^*$ is a commitment to the input bit x_i , and P and Q are public parameters of the signature scheme (generators of the underlying bilinear group). The value $h \leftarrow \mathbb{Z}_p^*$ is the blinding factor.

In contrast, *Verity-Auth* employs messages of length five. Specifically, the third component is a (blinded) random value $\vec{M}_i[3] = hv_iP$ for $v_i \leftarrow \mathbb{Z}_p^*$. Additionally, the two other components are the hashes of the secret key and the payload. The final message is given by:

$$\vec{M}_i = (h\text{com}_{x_i}, hP, hv_iP, hH_{\mathbb{G}_1^*}(\text{com}_{sk_i}), hH_{\mathbb{G}_1^*}(\text{payload}_i || \gamma_i))$$

Note that we concatenate the payload with a nonce γ_i , which is a random bit string of length determined by the security parameter κ . This is because of a technical detail in our security proof (see Sec. 7) which requires the fingerprint of the payload to form an equivocable commitment.

In addition, the user generates a ZK proof that the message is of the correct form. To be precise, this entails proving

- Correct commitment: $\pi_1 = \text{ZKPoK}\{r \in \mathbb{Z}_p \mid \text{com}_{x_i} = x_iP + rQ\}$
- Correct blinding: $\pi_2 = \text{ZK}\{\exists h \in \mathbb{Z}_p^* \mid \vec{M}_i[2] = hP \text{ and } \vec{M}_i[1] = h\text{com}_{x_i}\}$

The vector components corresponding to sk_i and payload_i need not be checked, as these will be validated in the verification phase.

On receiving the message, the authorizer first verifies it. For an honest user, we must have $x_i = x_i^A$, so the authorizer checks whether $\pi_{\vec{M}_i}$ shows a commitment to the ground truth x_i^A . If the check passes, the authorizer randomly samples $c_i \leftarrow \mathbb{Z}_p^*$ and issues a signature σ_i on a *modified* message, $\vec{M}_i^A$, where the third component is set to $\vec{M}_i^A[3] = c_i \vec{M}_i[3]$. The authorizer then sends c_i and σ_i to the user, who derives an unblinded and re-randomized signature σ'_i . Note that σ'_i is a signature on the message $\vec{M}_i' = (\text{com}_{x_i}, P, c_i v_i P)$. The user sets the token $\tau_i = H_{\mathbb{F}_p}(c_i v_i P)$, which will be used in the next phase. Here $H_{\mathbb{F}_p} : \mathbb{G}_1^* \mapsto \mathbb{F}_p$ is a hash function. c_i ensures the token remains uniformly random, while v_i masks the token and provides unlinkability – we will elaborate on this later.

Verity-Auth ensures input authorization with unlinkability via a new construction for blind signatures.

Noise Sampling Phase. The next step is to ensure that the user generates the correct noise sample. Previous works achieved this through interactive ZK protocols. While one might hope for a straightforward conversion of these protocols to a non-interactive one using the Fiat-Shamir transform, this approach is not applicable here. This limitation arises because here randomness is used not only for the probabilistic verification of deterministic statements, but also to generate probabilistic outputs [BGKW24]. As a result, a cheating prover (user) can carry out a poisoning attack without detection by simply using rejection sampling on the protocol. For instance, if a malicious user intends to report 1 regardless of their private input, they can resample protocol messages until they obtain an accepting transcript for an output of 1 (in expectation, the user needs to retry only $\lceil \frac{1}{\rho} \rceil$ times³).

We address this challenge by disentangling the noise sampling from the verification of its correctness. Specifically, we make the user sample noise *locally* in this phase. The task of verifying whether the user followed the protocol is delegated to the final verification phase – we design the sampling process in a way such that this verification is efficient.

In what follows, we elaborate on the local noise sampling process. Recall that RR_ρ involves flipping the private bit x_i with some probability ρ . This implies that the required noise is essentially a random sample drawn

³Even if the malicious user's true input is 0 and they follow the protocol honestly, the final response is 1 with probability ρ , implying the expected number of retries is $\lceil \frac{1}{\rho} \rceil$.

from $Ber(\rho)$. To generate a sample $b_{\rho,i} \sim Ber(\rho)$, i.e., a biased bit, proceed as follows: sample a uniform number $r \in [0, 1]$ and then output heads if $r \leq \rho$ and tails otherwise. This approach can be implemented with statistical error at most 2^{-k} by discretizing the sampling of r . Specifically, one can sample k bits uniformly at random, interpret these bits as the binary expansion of r , and then perform the comparison. Based on this observation, a user can locally sample the biased bit by generating uniform random bits with the aid of a PRF. In particular, we use the Legendre PRF for this purpose – $\mathcal{F}(j, K_i)$ gives us the j -th bit b_{ij} where K_i is the key of the PRF for user $\mathcal{U}_i$. This choice offers a two-fold advantage. First, the output domain of the Legendre PRF is binary, providing uniform random bits directly. Second, the evaluation of Legendre PRFs can be verified efficiently, as explained later.

The remaining detail is to discuss the key K_i of the PRF. Specifically, we need to ensure the following properties:

- K_i has to be kept secret from the server. This is to ensure that the noise ($b_{\rho,i}$) is hidden, else it would violate DP.
- K_i cannot be generated completely by the user. This is because a malicious user can select K_i strategically via rejection sampling to force the noisy bit to any value of their choosing.

We resolve this by splitting the key into two parts as follows:

$$K_i = \underbrace{sk_i}_{\text{User's secret}} + \underbrace{\tau_i}_{\text{Authorizer's token}} \quad (2)$$

The first component is a secret sk_i chosen by the user which ensures the privacy of the noise added. The second component is the token $\tau_i = H_{\mathbb{F}_p}(c_i v_i)$ derived in the previous phase. Recall that for the token, c_i is a random value chosen by the authorizer which prevents the user from influencing the process of key generation. Once $b_{\rho,i}$ is generated, the input bit can be flipped as:

$$y_i = (1 - x_i)b_{\rho,i} + x_i(1 - b_{\rho,i}) \quad (3)$$

Verity-Auth samples noise non-interactively by having the users locally evaluate a PRF on a *well-formed* key.

Verification of Computation Phase. In the final phase, the user communicates the noisy response y_i and the payload payload_i to the server. However, since *Verity-Auth* must account for malicious users, the server needs to verify the correctness of the user's computation. To this end, the user $\mathcal{U}_i$ also sends a set of ZK proofs, along with the unblinded signature σ'_i on the message $\vec{M}_i$ and the nonce γ_i . Specifically, the server needs to verify two pieces of computation:

1. *Evaluation of RR_ρ .* To prevent poisoning attacks that could involve tampering with the randomizer, the server must verify that RR_ρ was evaluated correctly. This involves two steps: (1) verifying the correct generation of uniform random bits b_{ij} 's using the PRF with the appropriate key K_i , and (2) ensuring the input bit was flipped correctly based on the sampled noise.

- *Evaluation of the PRF.* The typical way of expressing a computation for ZKPs is via an arithmetic circuit⁴. In other words, computations that can be expressed as small-depth arithmetic circuits lead to better performance for ZKPs. Our choice of the Legendre PRF is ZKP-friendly. Specifically, for a public quadratic non-residue n_{ij} , the PRF evaluation $b_{ij} = \mathcal{F}(j, K_i)$ in our protocol can be efficiently proven in ZK using the following relation with a simple arithmetic circuit

⁴Arithmetic circuits use addition and multiplication gates over a finite field.

$$\mathcal{R}_{b_{ij}} = \left\{ ((\text{com}_{\text{sk}_i}, b_{ij}), (\text{sk}_i, w_{ij})) \left| \begin{array}{ll} \text{sk}_i = \text{Open}(\text{com}_{\text{sk}_i}), & (4) \\ b_{ij}(1 - b_{ij}) = 0, & (5) \\ w_{ij}^2 - ((1 - b_{ij})n_{ij} + b_{ij})(K_i + j) = 0 & (6) \\ \text{where} & \\ K_i = \text{sk}_i + H_{\mathbb{F}_p}(\vec{M}_i[3]) & (7) \\ w_{ij} = (1 - b_{ij}) \cdot \sqrt{n_{ij}(K_i + j)} + b_{ij} \cdot \sqrt{K_i + j} & (8) \end{array} \right. \right\}$$

In the above relation, Eqs. (4) and (7) ensure that the correct key is used, while Eqs. (5), (6) and (8) demonstrate the correctness of the Legendre symbol computation. It is easy to see that multiple proofs of PRF evaluations on the same key K_i can be efficiently merged into a single ZK proof $\pi_{i,1}$ by concatenating their circuits.

- *Input bit flipping.* Recall, that the LDP randomizer, RR_ρ , flips the input bit based on the Bernoulli sample, $b_{\rho,i}$. The standard process of sampling $b_{\rho,i}$ involves a comparison operation, which is costly under ZKPs because it is a non-arithmetic operation. To address this challenge, we choose a ZKP-friendly noise distribution. Specifically, we set the bias of the Bernoulli distribution to be a power of $\frac{1}{2}$, i.e., $\rho = \frac{1}{2^k}$ where $k = \lceil \log_2(1 + e^\epsilon) \rceil$. In this way, the Bernoulli sample can be generated by a simple product of all the k uniform random bits $b_{\rho,i} = \prod_{j=1}^k b_{ij}$ and thereby, avoid the costly non-arithmetic comparison operation. The correct flipping of the input bit can then be proven in a ZKP π_{y_i} for the following relation:

$$\mathcal{R}_{y_i} = \left\{ ((\text{com}_{x_i}, y_i), (x_i, b_{\rho,i})) \left| \begin{array}{l} b_{\rho,i} = \prod_{j=1}^k b_{ij}, \\ y_i = (1 - x_i)b_{\rho,i} \\ \quad + x_i(1 - b_{\rho,i}) \end{array} \right. \right\} \quad (9)$$

The above proof, π_{y_i} , together with the proof of correct evaluation of the PRF, $\pi_{i,1}$, guarantees the correct evaluation of RR_ρ , thereby preventing randomizer tampering.

2. *Use of authorized input.* The final task is to verify that the user $\mathcal{U}_i$ computed on an authorized input, i.e., the input bit used for evaluating RR_ρ was checked by the authorizer $\mathcal{A}$. This check prevents input tampering attack. To accomplish this, the user produces a signature and message pair $(\sigma'_i, \vec{M}'_i)$ and a ZKP π_{x_i} such that:

- A ZKP showing that the message $\vec{M}'_i$ corresponds to a commitment to the flipped bit which is given by:

$$\pi_{x_i} = \text{ZK}\{\text{Open}(\text{com}_{x_i}) = \text{Open}(\vec{M}'_i[1])\} \quad (10)$$

where com_{x_i} is as defined in Eq. 9.

- $(\sigma'_i, \vec{M}'_i)$ is a valid signature issued by the authorizer, i.e., σ'_i can be verified using the authorizer's public key $\text{pprm}_{\mathcal{A}}$.

Thus, the ZK proof π_{x_i} ties the authorizer's signature to the user's computation. Due to the unforgeability of the signature, the user can only pass the above check if they used an authorized input $(\vec{M}'_i[1])$ in their computation.

To recall, the server obtains $y_i, \text{payload}_i$, along side signature σ'_i , ZK proofs $\pi_i = (\pi_{i,1}, \pi_{y_i}, \pi_{x_i})$, the message $\vec{M}'_i$ and the nonce γ_i from user $\mathcal{U}_i$. The server accepts $\mathcal{U}_i$'s response only if (1) all the proofs are valid, (2) σ'_i is a valid signature on $\vec{M}'_i$, and (3) the message contains the correct payload and the secret key registered during the setup phase, i.e.,

$$\vec{M}'_i[4] = H_{\mathbb{G}_1^*}(\text{com}_{\text{sk}_i}) \text{ and } \vec{M}'_i[5] = H_{\mathbb{G}_1^*}(\text{payload}_i || \gamma_i)$$

Note that $\mathcal{U}_i$'s interactions with the authorizer and the server remain unlinkable because:

- σ'_i is re-randomized which is unlinkable by the property of the EQS blind signature.
- The mask v_i is unknown to the server/authorizer and thereby prevents linking of the signatures via the token τ_i .

Verity-Auth provides unlinkability, assuming there are no side-channel attacks, such as timing attacks. This can be enforced by anonymizing user messages via shuffling before sending them to the server [TWM⁺24, vdHLZZ15].

Verity-Auth ensures efficient verification through a ZKP-friendly implementation.

5 *Verity*: Without Authorized Input

In this section, we explore the scenario where access to an authorized input is *unavailable*. Hence, a malicious user is free to lie and tamper with their input. Consequently, the only preventive measure against poisoning is ensuring the honest generation of randomness. However, a challenge arises – if the user can observe the noise sample in the clear, even if it is honestly generated, they could still mount arbitrary attacks by maliciously setting their input *based on* the generated sample. For an illustration, consider a malicious user who wishes to report the value 1. If the generated noise sample is $b_{i,\rho} = 1$ (i.e., the input bit will be flipped), the user set their input $x_i = 1$, and they set $x_i = 0$ otherwise. We tackle this challenge with *Verity*. The key idea is to restrict the users to generating only an *encrypted* noise sample. This ensures that any tampering with the inputs remain *independent* of the noise, providing the strongest protection in the absence of authorized inputs.

5.1 Technical Intuition

In *Verity*, users generate encrypted noise sample via a HE scheme. Specifically, users homomorphically evaluate the PRF to generate a sequence of uniform random bits, which are then converted to a Bernoulli sample. In particular, *Verity* uses a (2,2)-threshold HE scheme where the secret key is shared between both the user and the server. This restricts the information that can be decrypted by both parties – user $\mathcal{U}_i$ should not be able to decrypt the response y_i while the server should not be able to decrypt anything beyond the final response. Now the main task is to efficiently verify this computation. A natural solution is to rely on ZK proofs [KVMpAH24] or homomorphic MACs [BF11, FGP14]. In this approach, generating the ZK proof (or MAC) is computationally expensive but verifying the proof is efficient, making it suitable for settings where the verifier is resource-constrained. A typical example is verifiable outsourced computation, where the resource-constrained user outsources a computation-heavy task to an untrusted server, which is one of the most common use cases for HE schemes. However, the situation is reversed in our setting – the user is tasked with performing the homomorphic computation, which makes the above solution ill-suited. *Verity* solve this by taking a completely different approach to verification by leveraging the deterministic nature of the HE ciphertexts. The server replicates the entire computation on its side and checks that the generated ciphertexts match. If the user behaves honestly, the server’s computation should yield exactly the same ciphertext as the user’s. By shifting the computational load to the resource-heavy server, *Verity*, thus, takes advantage of the architectural constraints of our setting.

5.2 Workflow

Verity consists of two rounds, preceded by a one-time setup phase. In particular, it does not include the input authorization round as in *Verity-Auth*. The full protocol is outlined in Fig. 4.

Setup Phase. The one-time setup phase is used to initialize the keys for the threshold HE scheme and the PRFs. First, the server and each user $\mathcal{U}_i$ jointly sample the parameters of the HE scheme $(pk_i^{\text{HE}}, sk_{i,\mathcal{U}}^{\text{HE}}, sk_{i,\mathcal{S}}^{\text{HE}}) \leftarrow \text{HE.GenParam}(1^\kappa, 2, 2)$ where $sk_{i,\mathcal{S}}^{\text{HE}}$ and $sk_{i,\mathcal{U}}^{\text{HE}}$ are the secret keys of the server and $\mathcal{U}_i$, respectively. As

Parameters. ϵ - Privacy parameter; κ - Security parameter

- **Setup Phase.**

Authorizer:

- Generates the parameters from the blind signature scheme $(\text{pprm}_{\mathcal{A}}, \text{privprm}_{\mathcal{A}}) \leftarrow \text{EQS.GenParam}(1^\kappa)$

User: Each user $\mathcal{U}_i$

- Samples $\text{sk}_i \leftarrow \mathbb{F}_p$, sends a commitment $\text{com}_{\text{sk}_i} \leftarrow \text{Com}(\text{sk}_i)$ and proof $\pi_{\text{sk}_i} = \text{ZKPoK}\{\text{sk}_i \mid \text{sk}_i = \text{Open}(\text{com}_{\text{sk}_i})\}$ to the server

Server: For each user $\mathcal{U}_i$

- Rejects $\mathcal{U}_i$ if π_{sk_i} does not verify

- **Input Authorization Phase.**

User: Each user $\mathcal{U}_i$

- Prepares the signature request using x_i , sk_i and payload: $(\vec{M}_i, \pi_{\vec{M}_i}, \gamma_i, \text{st}_i) \leftarrow \text{EQS.SignRequest}(x_i, \text{payload}_i, \text{com}_{\text{sk}_i}, \text{pprm}_{\mathcal{A}})$
- Stores the state information st_i with itself

- Sends the message $\vec{M}_i$ and the associated proof $\pi_{\vec{M}_i}$ to the authorizer: $\mathcal{U}_i \xrightarrow{\vec{M}_i, \pi_{\vec{M}_i}} \mathcal{A}$

Authorizer: For each user $\mathcal{U}_i$

- Verifies the proof: $b = \text{EQS.VerifyMsg}(\vec{M}_i, x_i^{\mathcal{A}}, \pi_{\vec{M}_i})$
- If $b = 1$, issues a blind signature on the message and sends it to the user: $\sigma_i, c_i \leftarrow \text{EQS.Sign}(\vec{M}_i, \text{sk}_{\mathcal{A}}), \mathcal{A} \xrightarrow{\sigma_i, c_i} \mathcal{U}_i$

User: Each user $\mathcal{U}_i$

- Unblinds the signature to obtain a new signature and token τ_i : $(\vec{M}'_i, \sigma'_i) \leftarrow \text{EQS.Unblind}(\sigma_i, \text{st}_i, c_i), \tau_i = H_{\mathbb{F}_p}(\vec{M}'_i[3])$

- **Noise Sampling Phase.**

User: Each user $\mathcal{U}_i$

- Sets $k = \lceil \log_2(1 + e^\epsilon) \rceil$ and the Legendre PRF key $K_i = \text{sk}_i + \tau_i$
- For all $j \in [k]$, computes $b_{ij} = \mathcal{F}(K_i, j)$
- Computes the biased bit (sample of noise) $b_{\rho,i} = \prod_{j=1}^k b_{ij}$ and the noisy response $y_i = b_{\rho,i} \cdot (1 - x_i) + (1 - b_{\rho,i}) \cdot x_i$

- **Verification of Computation Phase.**

User: Each user $\mathcal{U}_i$

- Selects quadratic non-residues n_{ij} for $j \in [k]$, and generates a zero knowledge proof for showing

$$\star \text{ Correct PRF evaluation } \pi_{i,1} = \left(\{n_{ij}\}, \text{ZK} \left\{ \begin{array}{l} \text{sk}_i = \text{Open}(\text{com}_{\text{sk}_i}), \\ \forall j \in [k], b_{ij}(1 - b_{ij}) = 0 \text{ AND } w_{ij}^2 - ((1 - b_{ij})n_{ij} + b_{ij})(K_i + j) = 0 \text{ where} \\ K_i = \text{sk}_i + H_{\mathbb{F}_p}(\vec{M}'_i[3]), w_{ij} = \sqrt{n_{ij}(K_i + j)} \text{ if } b_{ij} = 0, \text{ or } w_{ij} = \sqrt{K_i + j} \end{array} \right\} \right)$$

$$\star \text{ Correct flipping of the authorized input bit: } \pi_{i,2} = \text{ZK}\{y_i = b_{\rho,i} \cdot (1 - x_i) + (1 - b_{\rho,i})x_i \text{ where } x_i = \text{Open}(\vec{M}'_i[1])\}$$

- Sends the noisy response y_i , payload_i , $\pi_i = (\pi_{i,1}, \pi_{i,2})$, message, signature, nonce to the server: $\mathcal{U}_i \xrightarrow{y_i, \text{payload}_i, \pi_i, \vec{M}'_i, \sigma'_i, \gamma_i} \mathcal{S}$

Server: For each user $\mathcal{U}_i$

- Verifies the proof π_i using the commitment com_{sk_i} received in the setup phase, reject $\mathcal{U}_i$ otherwise
- Verifies the signature σ'_i as $b \leftarrow \text{EQS.VerifyToken}(\sigma', \vec{M}'_i, \text{pprm}_{\mathcal{A}}, \text{com}_{\text{sk}_i}, \text{payload}_i, \gamma_i)$, reject $\mathcal{U}_i$ if invalid

Figure 3: *Verity-Auth*: Verifiable randomized response where the users have authorized inputs.

Parameters. ϵ - Privacy parameter; κ - Security parameter

• **Setup Phase.**

Server and User:

- The server $\mathcal{S}$ and user $\mathcal{U}_i$ jointly generates the keys of the threshold HE scheme: $(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{U}}^{\text{HE}}, \text{sk}_{i,\mathcal{S}}^{\text{HE}}) \leftarrow \text{HE.GenParam}(1^\kappa)$ where $\text{sk}_{i,\mathcal{S}}^{\text{HE}}$ and $\text{sk}_{i,\mathcal{U}}^{\text{HE}}$ is known only to the server and the user $\mathcal{U}_i$
- Initialize $\text{cnt}_i = 0$

Server:

- Samples its share of the secret key $\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}$ for user $\mathcal{U}_i$'s PRF as $\text{sk}_{i,\mathcal{S}}^{\mathcal{F}} \leftarrow \mathbb{F}_p$
- Encrypts the sampled key and generates a ZK proof of the correctness of the encryption

$$\text{ct}_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}} \leftarrow \text{HE.Enc}(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{S}}^{\mathcal{F}}), \pi_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}} \leftarrow \text{ZKPoK}\{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}} \in \mathbb{F}_p \mid \text{ct}_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}} = \text{HE.Enc}(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{S}}^{\mathcal{F}})\}$$

- Sends the generated ciphertext and ZK proof to user: $\mathcal{U}_i, \mathcal{S} \xrightarrow{\text{ct}_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}}, \pi_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}}} \mathcal{U}_i$

User: Each user $\mathcal{U}_i$:

- Verifies the proof $\pi_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}}$ and aborts in case of the check fails
- Samples its share of the secret key for the PRF $\text{sk}_{i,\mathcal{U}}^{\mathcal{F}} \leftarrow \mathbb{F}_p$
- Encrypts the sampled key and generates a ZK proof of the correctness of the encryption

$$\text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}} \leftarrow \text{HE.Enc}(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}}), \pi_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}} \leftarrow \text{ZKPoK}\{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}} \in \mathbb{F}_p \mid \text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}} = \text{HE.Enc}(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}})\}$$

- Sends the generated ciphertext and ZK proof to the server $\mathcal{U}_i \xrightarrow{\text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \pi_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}} \mathcal{S}$

Server: For each user $\mathcal{U}_i$:

- Continues if $\pi_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}$ verifies, reject otherwise.

• **Noise Sampling Phase.**

User: Each user $\mathcal{U}_i$

- Encrypts the input bit x_i and generate a ZK proof of the correctness of the encryption

$$\text{ct}_{x_i} \leftarrow \text{HE.Enc}(\text{pk}_i^{\text{HE}}, x_i); \pi_{x_i} \leftarrow \text{ZKPoK}\{x_i \in \{0, 1\} \mid \text{ct}_{x_i} = \text{HE.Enc}(\text{pk}_i^{\text{HE}}, x_i)\}$$

- Sets $k \leftarrow \lfloor \log_2(1 + e^\epsilon) \rfloor$ and let $f_{\text{cnt}_i}(\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}}, x_i) = x_i + (1 - 2x_i) \prod_{j=1}^k \mathcal{F}(\text{sk}_{i,\mathcal{S}}^{\mathcal{F}} + \text{sk}_{i,\mathcal{U}}^{\mathcal{F}} + \text{cnt}_i, j)$
- Homomorphically evaluates f_{cnt_i} to generate the ciphertext of the response: $\text{ct}_{y_i} \leftarrow \text{HE.Eval}(\text{pk}_i^{\text{HE}}, \text{ct}_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}}, \text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \text{ct}_{x_i}, f_{\text{cnt}_i})$
- Partially decrypts ct_{y_i} using the secret key $\text{sk}_{i,\mathcal{U}}^{\text{HE}}$ and generate a ZK proof showing the correctness of this step

$$d_{i,\mathcal{U}} \leftarrow \text{HE.PartialDec}(\text{sk}_{i,\mathcal{U}}^{\text{HE}}, \text{ct}_{y_i}), \pi_{y_i} \leftarrow \text{ZKPoK}\{\text{sk}_{i,\mathcal{U}}^{\text{HE}} \mid d_{i,\mathcal{U}} = \text{HE.PartialDec}(\text{sk}_{i,\mathcal{U}}^{\text{HE}}, \text{ct}_{y_i})\}$$

- Updates $\text{cnt}_i = \text{cnt}_i + 1$
- Sends all the ciphertexts, the partial decryption of the response and the ZK proofs to the server: $\mathcal{U}_i \xrightarrow{\text{ct}_{y_i}, \text{ct}_{x_i}, d_{i,\mathcal{U}}, \pi_{x_i}, \pi_{y_i}} \mathcal{S}$

• **Verification of Computation Phase.**

Server: For each user $\mathcal{U}_i$, the server

- Verifies the proof π_{x_i} on ct_{x_i} and reject in case of an invalid proof.
- Re-computes the ciphertext for the response: $\text{ct}'_{y_i} = \text{HE.Eval}(\text{pk}_i^{\text{HE}}, \text{ct}_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}}, \text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \text{ct}_{x_i}, f_{\text{cnt}_i})$.
- Continue only if $\text{ct}_{y_i} = \text{ct}'_{y_i}$ and π_{y_i} verifies. Rejects otherwise.
- Generates its decryption share for ct_{y_i} and combines $(d_{i,\mathcal{U}}, d_{i,\mathcal{S}})$ for full decryption:

$$d_{i,\mathcal{S}} \leftarrow \text{HE.PartialDec}(\text{sk}_{i,\mathcal{S}}^{\text{HE}}, \text{ct}_{y_i}); y_i \leftarrow \text{HE.Combine}(\text{pk}_i^{\text{HE}}, d_{i,\mathcal{U}}, d_{i,\mathcal{S}})$$

- Updates $\text{cnt}_i = \text{cnt}_i + 1$

Figure 4: *Verity*: Verifiable randomized response where the users do not have any authorized input.

elaborated in the following para, the secret key for the PRF associated with $\mathcal{U}_i$ is also split between the two parties. Specifically, the server and $\mathcal{U}_i$, each sample their respective shares of the PRF secret key – $\text{sk}_{i,S}^{\mathcal{F}} \in \mathbb{F}_p$ and $\text{sk}_{i,\mathcal{U}}^{\mathcal{F}} \in \mathbb{F}_p$, respectively, for $\mathcal{U}_i$'s PRF. Since both parties need to homomorphically evaluate the PRF, the key shares must be encrypted and exchanged between them. Additionally, both parties generate a ZK proof of the correctness of this encryption and send it to each other for verification. To be precise, the server sends the following to $\mathcal{U}_i$:

$$\begin{aligned} \text{ct}_{\text{sk}_{i,S}^{\mathcal{F}}} &\leftarrow \mathbf{HE.Enc}(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}}) \\ \pi_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}} &\leftarrow \text{ZKPoK}\{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}} \in \mathbb{F}_p \mid \text{ct}_{\text{sk}_{i,S}^{\mathcal{F}}} = \mathbf{HE.Enc}(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}})\} \end{aligned}$$

Similarly, user $\mathcal{U}_i$ sends $(\text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \pi_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}})$ to the server. Finally, both the parties initialize a counter $\text{cnt}_i = 0$ which tracks the number of messages exchanged between them.

Noise Sampling Phase. The sampling process is the same as *Verity-Auth* with some modifications to accommodate homomorphic evaluation. In particular, $\mathcal{U}_i$ uses a PRF key

$$K_i = \text{sk}_{i,S}^{\mathcal{F}} + \text{sk}_{i,\mathcal{U}}^{\mathcal{F}} + \text{cnt}_i$$

Here $\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}$ ensures that the sampled noise is kept private from the server, while $\text{sk}_{i,S}^{\mathcal{F}}$ prevents the user $\mathcal{U}_i$ from generating the PRF key on their own maliciously. These two components together ensure that the key is well-formed, as discussed in Sec. 4. Additionally, the counter cnt_i ensures that a different key is used for each message, providing a fresh noise sample every time.⁵ Re-arranging the equations, the noisy response is then given by:

$$\begin{aligned} y_i &= f_{\text{cnt}_i}(\text{sk}_{i,S}^{\mathcal{F}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}}, x_i) \\ &= x_i + (1 - 2x_i) \prod_{j=1}^k \mathcal{F}(\text{sk}_{i,S}^{\mathcal{F}} + \text{sk}_{i,\mathcal{U}}^{\mathcal{F}} + \text{cnt}_i, j) \end{aligned}$$

The user $\mathcal{U}_i$ encrypts their input x_i as ct_{x_i} and generates a ZK proof of correctness of the encryption, π_{x_i} . $\mathcal{U}_i$ is now ready to compute the ciphertext of the response as:

$$\text{ct}_{y_i} \leftarrow \mathbf{HE.Eval}(\text{pk}_i^{\text{HE}}, \text{ct}_{\text{sk}_{i,S}^{\mathcal{F}}}, \text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \text{ct}_{x_i}, f_{\text{cnt}_i})$$

At this point, the user can only *partially* decrypt the response (equivalently the noise sample $b_{\rho,i}$) because of *Verity*'s usage of threshold HE scheme. This ensures that the noise sample remains hidden from the user in *Verity*. The user proceeds with the partial decryption of the response, and additionally creates a ZK proof of correctness of this decryption:

$$\begin{aligned} d_{i,\mathcal{U}} &\leftarrow \mathbf{HE.PartialDec}(\text{sk}_{i,\mathcal{U}}^{\text{HE}}, \text{ct}_{y_i}) \\ \pi_{y_i} &\leftarrow \text{ZKPoK}\{\text{sk}_{i,\mathcal{U}}^{\text{HE}} \mid d_{i,\mathcal{U}} = \mathbf{HE.PartialDec}(\text{sk}_{i,\mathcal{U}}^{\text{HE}}, \text{ct}_{y_i})\} \end{aligned}$$

Finally, the user updates the counter $\text{cnt}_i = \text{cnt}_i + 1$ and sends $(\text{ct}_{x_i}, \text{ct}_{y_i}, d_{i,\mathcal{U}}, \pi_{x_i}, \pi_{y_i})$ to the server for verification.

Verification of Computation Phase. The server first verifies the proof π_{x_i} on ct_{x_i} and aborts in case of an invalid proof. Next, the server re-computes the ciphertext:

$$\text{ct}'_{y_i} = \mathbf{HE.Eval}(\text{pk}_i^{\text{HE}}, \text{ct}_{\text{sk}_{i,S}^{\mathcal{F}}}, \text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \text{ct}_{x_i}, f_{\text{cnt}_i})$$

Since $\mathbf{HE.Eval}$ is deterministic, we must have $\text{ct}_{y_i} = \text{ct}'_{y_i}$ for an honest user. Additionally, the proof of partial decryption π_{y_i} must be valid. If both conditions are satisfied, the server proceeds to decrypt ct_{y_i} . To do this, $\mathcal{S}$ generates its decryption share $d_{i,S}$ and combines $(d_{i,\mathcal{U}}, d_{i,S})$ for full decryption as:

⁵This is not required for *Verity-Auth* since the key includes a fresh token τ_i for each message. Also, a public nonce, such as the timestamp, can be used in place of a counter.

$$d_{i,S} \leftarrow \text{HE.PartialDec}(\text{sk}_{i,S}^{\text{HE}}, \text{ct}_{y_i})$$

$$y_i \leftarrow \text{HE.Combine}(\text{pk}^{\text{ct}_{y_i}}, d_{i,U}, d_{i,S}).$$

Verity efficiently verifies HE computation by leveraging the deterministic nature of the HE ciphertexts.

6 Drop Out Attacks

Here, we analyze the impact of drop out attacks on the two solutions presented so far. For *Verity-Auth*, although the input itself cannot be tampered with, a malicious user could selectively drop out *based on the sampled noise*. This attack is possible even when the noise is sampled honestly, as users can observe the noise in the clear before reporting. For instance, consider a malicious user $\mathcal{U}_i$ with true input $x_i^A = 1$. If $\mathcal{U}_i$ wants to report a 0, they could drop out if the sampled noise is $b_{i,\rho} = 1$ (i.e., the input bit would be flipped).

This attack can be prevented by combining both of our proposed protocols to ensure both authorized input as well as honestly generated, *encrypted* noise. In particular, we start with the protocol *Verity* and introduce the following changes. First, the protocol is augmented with an additional phase of input authorization which is identical to that of *Verity-Auth*. Second, the key of the Legendre PRF is modified to match that of *Verity-Auth*. In particular, we no longer require $\text{sk}_{i,S}^{\mathcal{F}}$ and cnt_i ; instead we have $K_i = \text{sk}_{i,S}^{\mathcal{F}} + \tau_i$ as in *Verity-Auth*. Finally, in the verification phase, each user provides additional information—the signature and message pair $(\sigma'_i, \vec{M}'_i)$, along with an additional ZKP: $\pi_{x_i} \leftarrow \text{ZKPoK}\{z \in \{0, 1\} \mid z = \text{Open}(\vec{M}'_i[1]) \text{ and } \text{ct}_{x_i} = \text{HE.Enc}(\text{pk}_i^{\text{HE}}, z)\}$. The goal of this information is to prove that RR was evaluated under HE on the authorized input. Thus, a malicious user can drop out *only* on its input (which cannot be tampered with) and public parameters (ϵ determines the probability of flipping), providing the strongest guarantee that can be achieved in this context. We call this protocol as *Verity-Auth*⁺ (Fig. 8). For *Verity*, the sampled noise is encrypted, preventing the adversary from exploiting it. Since a malicious user can change their input arbitrarily, they gain no advantage by dropping out.

7 Security Analysis

We analyze security in the real-ideal world security model [Ode09b], assuming a malicious adversary. Fig. 5 and Fig. 6 show the ideal functionalities for *Verity-Auth* and *Verity*, respectively. The functionality in Fig. 6 evaluates RR honestly on the inputs that are *chosen* by the users, and outputs all the responses. In contrast, in Fig. 5, the input is authorized but the adversary has the ability to drop out based on the randomness.

We analyze *Verity-Auth* in two cases: when the authorizer is honest or when it is controlled by a malicious adversary.

***Verity-Auth* – Honest Authorizer.** In this setting, ZKP ensures that malicious users can only produce valid proofs for outputs generated honestly using RR on the authorized input. Since the PRF key is jointly generated by the user and the authorizer, the user cannot control or predict the randomness. Their only option is to drop out if dissatisfied with the output. This is formalized as:

Theorem 4. *Model $\mathcal{H}_{\mathbb{G}_1^*}$ as a random oracle. Assume $\mathcal{F}$ is a PRF, Pedersen scheme is a commitment and our proofs are ZKPs. Let a malicious adversary control some parties but not the authorizer. Then *Verity-Auth* achieves the ideal functionality of Fig. 5.*

***Verity-Auth* – Corrupted Authorizer.** We do not prevent an authorizer colluding with a user from providing them with multiple signed bits – the whole premise of *Verity-Auth* is to leverage an authorizer to prevent malicious user behavior. In such cases, the adversary cannot access honest users' payloads or

Parameters. ϵ - Privacy parameter User's Input. Each user $\mathcal{U}_i$ inputs a bit x_i and chooses payload_i Authorizer's Input. Authorizer inputs the ground truth x_i^A for every user $\mathcal{U}_i$ Functionality. • Set $k \leftarrow \lfloor \log_2(1 + e^\epsilon) \rfloor$ and $\rho = \frac{1}{2^k}$ • Let $\mathcal{I}$ be the set of users whose input x_i matches the x_i^A from the authorizer $\mathcal{A}$ • For every user $\mathcal{U}_i \in \mathcal{I}$ – Generate a sample of $b_{i,\rho} \sim \text{Ber}(\rho)$ – Set $y_i = (1 - x_i^A)b_{i,\rho} + (1 - b_{i,\rho})x_i^A$ – If $\mathcal{U}_i$ is corrupt, send them y_i and give them an opportunity to drop out • Send $(y_i, \text{payload}_i)_i$ s.t. $\mathcal{U}_i$ hasn't dropped to the server $\mathcal{S}$
--

Figure 5: Ideal Functionality for *Verity-Auth*

Parameters. ϵ - Privacy parameter User's Input. Each user $\mathcal{U}_i$ inputs a bit x_i. Functionality. • Set $k \leftarrow \lfloor \log_2(1 + e^\epsilon) \rfloor$ and $\rho = \frac{1}{2^k}$ • For every user $\mathcal{U}_i$ – Generate a sample of $b_{i,\rho} \sim \text{Ber}(\rho)$ – Set $y_i = (1 - x_i)b_{i,\rho} + (1 - b_{i,\rho})x_i$ – Send y_i to the server $\mathcal{S}$
--

Figure 6: Ideal Functionality for *Verity*

alter their submissions, but can only drop them. However, even if the authorizer is malicious, it cannot compromise the privacy of honest users. This holds true even if both the authorizer and server are colluding, as stated in as Thm 5. The private information we want to protect is the join of payload_i and x_i since the authorizer knows all x_i . Intuitively what we want to prove is that the input authorization that our protocol relies on, does not enable the authorizer to fingerprint the user contribution in a way that enables the server to link it to a payload. While the authorizer and server might have side channels, we argue that our construction introduces no *additional* linking. To model this formally, we assume that the responses are submitted to the server in a shuffled order.

Theorem 5. *Assume the ZKPs are Perfect Special Honest-Verifier Zero-Knowledge i.e., they can be faked by a simulator, $H_{\mathbb{G}_1^*}$ is a random oracle, $\mathcal{F}$ is a PRF and EQS is a blind signature scheme. Suppose a malicious adversary controls the authorizer, the server and a subset of users. Then Verity-Auth achieves the ideal functionality of Fig. 5.*

Furthermore, if the collection of messages to the server in each round are shuffled (by an honest third party or other equivalent means) before being given to the server. Then the protocol implements that functionality with the servers outputs being shuffled.

The blind signatures ensure unlinkability between signing requests and signatures, preventing any additional communication between the authorizer and the server

Verity. *Verity* prevents randomness-dependent tampering with the input for two reasons. First, the randomness is encrypted. Second, the adversary cannot attack homomorphically, as the output ciphertext results from a homomorphic evaluation of the expected fixed functionality.

Theorem 6. *Assume our proofs are ZKPs, $\mathcal{F}$ is a PRF and the HE scheme is semantically secure. Verity achieves the ideal functionality of Fig. 6 with malicious security.*

8 Robustness Analysis

With Authorized Input. We start by analyzing *Verity-Auth* in the case where there are no drop-outs, i.e., all n users are required to submit a valid response.

Lemma 7 (*Verity-Auth* Without drop-out). $\hat{s}$ has ℓ_1 error $\Theta(\frac{\sqrt{n}}{\epsilon})$ in *Verity-Auth*, regardless of the number of malicious users.

Since *Verity-Auth* can prevent both input tampering and randomizer tampering, poisoning attacks are completely prevented in the absence of drop outs. The only error term, $\frac{\sqrt{n}}{\epsilon}$, is the inherent error introduced by the randomness required to satisfy *LDP* [BNO08, CSS12, DJW13].

Next, we analyze the case when the users can drop out.

Lemma 8 (*Verity-Auth*). *W.l.o.g* let $\{\mathcal{U}_i\}_{i \in [m]}$ be the m malicious users. Then an adversary can skew $\hat{s}$ by an ℓ_1 error $\tilde{O}(w(1 + \frac{1}{\epsilon}) + \frac{\sqrt{n}}{\epsilon})$, $w \in [m]$, with probability $\sum_{j=\max\{0, w+q-m\}}^{\min\{q, w\}} \binom{q}{j} \binom{m-q}{w-j} (1-\rho)^{m+2j-q-w} \rho^{q+w-2j}$ if dropping out based on the randomness, where $q = \sum_{i=1}^m x_i$. In particular, the malicious users can introduce the worst case error of $\tilde{O}(m(1 + \frac{1}{\epsilon}) + \frac{\sqrt{n}}{\epsilon})$ w.p. at most $\left(\frac{e^\epsilon}{1+e^\epsilon}\right)^q \left(\frac{1}{1+e^\epsilon}\right)^{m-q}$.

In *Verity-Auth*, malicious users can observe the sampled noise and can drop out based on that. Specifically, let $w \in \{0, m\}$ be the sum of the noisy responses of the malicious users. An adversary can then introduce error proportional to w . Recall that the strongest poisoning attack is when all the malicious users directly change their response (y_i) to be 1 (or 0 equivalently), which can introduce an error of $\tilde{O}(m(1 + \frac{1}{\epsilon}) + \frac{\sqrt{n}}{\epsilon})$ (Lemma 2). In our context, the adversary can still induce this worst case error when $w = m$, albeit with a significantly lower probability. Specifically, without any verifiability, malicious users can *always* lie about all m of their responses, and thereby carry out the worst-case attack with arbitrarily high probability. In contrast, in *Verity-Auth*, adversary can execute this attack only with probability bounded by that of obtaining $w = m$ by running RR on the authorized inputs. Thus, the effectiveness of the attack is now determined by both the randomness of RR and the distribution of the authorized inputs of the malicious users.

Lemma 9 (*Verity-Auth*⁺). *W.l.o.g* let $\{\mathcal{U}_i\}_{i \in [m]}$ be the m malicious users. Then an adversary can skew $\hat{s}$ by an ℓ_1 error of $\tilde{O}(q + \frac{\sqrt{n-m}}{\epsilon})$ with arbitrarily high probability, where $q = \sum_{i=1}^m x_i$.

By encrypting the noise, the adversary is restricted to drop out only based on its input (which cannot be tampered with) in *Verity-Auth*⁺. The adversary's influence is then bounded by the sum of the authorized inputs of the malicious users, q .

Without Authorized Input. The robustness of *Verity* is:

Lemma 10 (*Verity*). With m malicious users, an adversary can skew $\hat{s}$ by an ℓ_1 error of $\tilde{O}(m + \frac{\sqrt{n}}{\epsilon})$ with arbitrarily high probability.

In the absence of authorized inputs, the best one can do is limit the adversary to input poisoning attacks. This is precisely what *Verity* achieves — we observe that the resulting error aligns with the error from input poisoning attacks as in Lemma 3.

9 Implementation and Evaluation

We benchmarked both our protocols on an Intel Ice Lake CPU running at 2.6GHz with AVX512 instructions. We compute the Legendre PRF as $\mathcal{F}(x, K) = 2^{-1} \cdot (((K + x)^{(p-1)/2} \bmod p) + 1) \bmod p$.

9.1 *Verity-Auth*

Implementation. We used an 120-bit Legendre PRF modulus $p = (2^{16} + 2^{18} + 2^{19} + 2^{119}) \cdot 2 + 1$. As shown by [KKK20a, KKK20b], it takes $2^{89.7}$ time to recover the secret key when given 2^{20} Legendre PRF values. So, the user should rerun the setup phase and refresh its PRF key every $2^{20}/k$ sessions, where k is the number of PRF bits per protocol invocation. We instantiate the EQS signature using the Tate Pairing over curveFp254BNb [GJNB11], and we used Bulletproofs [BBB⁺18, rus23] to implement the ZK proofs for correct PRF and RR computation. Bulletproofs operates directly on committed inputs without a setup

Phase	Party	Description	Running Time / Communication Cost				
			$k = 2$ ($\epsilon = 1.1$)	$k = 3$ ($\epsilon = 2$)	$k = 4$ ($\epsilon = 2.8$)	$k = 5$ ($\epsilon = 3.5$)	$k = 6$ ($\epsilon = 4.2$)
Setup	Authorizer	Generate parameters	0.48ms / 0.86KB				
	User	Generate ZKPoK of sk_i	0.35ms / 0.38KB				
	Server	Verify ZKPoK of sk_i	0.60ms / -				
Input Authorization	User	Generate sign request	1.62ms / 1.12KB				
	Authorizer	Verify sign request & issue blind signature	2.05ms / 0.32KB				
	User	Unblind the signature	3.05ms / -				
Noise Sampling	User	Compute PRFs and RR bit	200 μ s / -	300 μ s / -	400 μ s / -	500 μ s / -	600 μ s / -
		Generate ZK proof of PRF and RR computation	5.22ms	8.37ms	9.15ms	9.92ms	15.33ms
			1.02KB	1.19KB	1.30KB	1.40KB	1.56KB
Verification	Server	Verify ZK proofs	0.90ms	1.38ms	1.44ms	1.50ms	2.26ms
		Verify signature	2.16ms				

Table 2: Results for *Verity-Auth* when the RR bit is computed using different numbers (k) of PRF evaluations. The protocol is independent of k until noise sampling. Communication cost is measured by outbound bandwidth, marked with a “-” if no outbound communication occurs.

Phase	Party	Description	Running Time / Communication Cost		
			$k = 2(\epsilon = 1.1)$	$k = 4(\epsilon = 2.8)$	$k = 6(\epsilon = 4.2)$
Setup	Server & User	Generate HE keys	21.41 s / 9126.88 MB	28.31 s / 9301.44 MB	35.16 s / 9476 MB
	Server	Sample and encrypt PRF key share	0.89 s / 78.44 MB	0.91 s / 80.22 MB	0.93 s / 82 MB
		Generate ZKPoK of encryption	1160.94 s / 3.17 KB		
	User	Verify Server’s ZKPoK	1233.00 s / -		
		Sample and encrypt PRF key share	0.89 s / 78.44 MB	0.91 s / 80.22 MB	0.93 s / 82 MB
		Generate ZKPoK of encryption	1160.94 s / 3.17 KB		
	Server	Verify User’s ZKPoK	1233.00 s / -		
Noise Sampling	User	Encrypt input bits	0.89 s / -	0.91 s / -	0.93 s / -
		Generate ZKPoK of encryption	1160.94 s / 3.17 KB		
		Homomorphic evaluation of PRF and RR bit	69.48 s / -	73.45 s / -	78.17 s / -
		Partial decryption	0.26 s / 2.02 MB		
		Generate ZKPoK of partial decryption	133.57 s / 1.76 KB		
Verification	Server	Verify ZKPoK of encryption	1233.00 s / -		
		Re-compute homomorphic evaluation	69.48 s / -	73.45 s / -	78.17 s / -
		Verify ZKPoK of partial decryption	117.0 s / -		
		Partial decryption and recover result	0.26 s / -		

Table 3: Results for *Verity* when the RR bit uses different numbers (k) of PRF evaluations. Steps with identical performance across k are highlighted in bold. $k = \{3, 4\}$ and $k = \{5, 6, 7, 8\}$ have identical costs because of the same complexity of the circuit for HE. Communication cost is measured by outbound bandwidth, marked with a “-” if no outbound communication occurs.

process. As a result, we no longer need a separate proof (π_{x_i} ; Eq. 10) showing the consistency between the bit used for evaluating RR and the authorized bit, and instead, the server verifies π_{y_i} (Eq. 9) directly using the commitment $\vec{M}_i[1]$. Additionally, we moved the ZKPoK of com_{sk_i} to the setup phase instead of sending it as part of $\pi_{b_{ij}}$ for every message.

Results. We report the mean of 100 runs in Table 2. *Verity-Auth* is independent of the number of PRF bits, k , up to the noise sampling phase. Our evaluation shows that *Verity-Auth* is very efficient: the setup phase takes < 1 ms for each party; the user spends < 5 ms during input authorization, while the authorizer spends 2 ms. The costliest step is to generate the ZK proof of correct PRF evaluations: for $k \in [2, 6]$, the prover’s (user) cost is ≤ 15 ms, and the verifier’s (server) cost is ≤ 4 ms. The communication costs are also low – the respective costs for the authorizer, user and the server are 2.3 KB, ≤ 5.02 KB and ≤ 3.58 KB. Our performance improves with lower ϵ since the flipping probability is given by $\rho = 1/2^k$.

Baseline. The closest prior work to ours is a concurrent work by Bontekoe et al. [BAT24] that also uses authorized inputs but in a weaker threat model (see Sec. 10). Reporting a single instance of verifiable RR takes each user 1.31s, which is 100× slower than *Verity-Auth*. Complete details are in App. D.

9.2 *Verity*

Implementation. Our $(2, 2)$ -threshold HE scheme is based on the BGV leveled homomorphic encryption scheme [BGV12] and is implemented using OpenFHE [BAB⁺22], and our ZK proofs are again implemented using Bulletproofs. In particular, partial decryption applies noise flooding to statistically hide the secret key $\text{sk}_{i,p}^{\text{HE}}$ from the output. We used a 40-bits Legendre PRF modulus $p = (2^{17} + 2^{19} + 2^{39}) \cdot 2 + 1$ to be compatible with existing HE implementations. Although p is much smaller than the one in *Verity-Auth*, the key recovery attack in [KKK20a] still requires roughly 2^{28} word operations when given 2^{10} PRF values. As a result, the user must refresh its secret key more frequently. Since p is an NTT-friendly prime, multiple Legendre PRFs and the corresponding Bernoulli bits can be evaluated in the “SIMD” slots in parallel. We set the RLWE ring degree to 2^{17} , the ciphertext modulus to 3104 bits, and the discrete Gaussian with parameter 3.2 for the RLWE secret and error distributions. Such parameters provide roughly 146 bits of classical security [APS15], and they support Bernoulli bits that are products of at most 8 PRF bits. To prove RLWE encryption and partial decryption in ZK, we express these algorithms as inner product constraints over the secret values as in [BGL⁺24]. Since the Bulletproofs implementation operates in a 252-bit group that is smaller than the HE ciphertext modulus and insufficient for expressing RLWE relations as is, we partition the RNS moduli into groups such that the product Q_j of moduli in each group j is small enough to prove RLWE relations modulo Q_j .

Results. Table 3 shows our results. For $k = 6$, the server and the user both take 2535s in the setup phase, where almost the entire time is spent on generating and verifying the ZK proof of encryption. The user’s online time is 1423.50s while that of the server is 1472.77s. Note that, with the same cost we can homomorphically generate up to 2^{13} many such Bernoulli bits with $k \leq 8$, thanks to the SIMD feature. For $k = 4$ and $k = 2$, $\mathcal{F}(x, K)$ has shallower circuits and requires smaller ciphertext moduli, resulting in less expensive HE setup and evaluation. On the other hand, the ZK proofs have the same cost as we use the same number of groups to partition the RNS moduli.

Baseline. A recent work [KVMpAH24] considers the alternative approach of using a ZKP to directly prove the correct homomorphic evaluation. However, even proving a *single* multiplication and a modulus switching using relatively small RLWE parameters requires 433s, which is roughly 1/3 of the online cost of *Verity* for each party. In fact, their RLWE parameters are insufficient to evaluate our circuits. The inefficiency stems from the numerous non-arithmetic operations, such as gadget decomposition and modulus switching, used in HE. Expressing these operations as ZK constraints results in a massive ZK statement, making it practically infeasible.

10 Related Work

A common point of difference with *all* prior work is that *Verity* is the first non-interactive protocol that offers the strongest protection where users lack authorized inputs, by keeping the randomness encrypted. Additionally, our work is the first to analyze drop outs for verifiable DP. Existing works on verifiable RR [AJL03, KAGN99, KCY21, SXZ23] are interactive in nature. Biswas et al. [BC23] proposed an interactive protocol for the Binomial mechanism. Moran et. al. proposed a human-centric solution using physical envelopes [MN06]. Furthermore, these works do not consider authorized inputs. A few prior works [MGSB22, MDMT23, BAT24] have considered authorized inputs, but they either fail to provide unlinkability [MGSB22] or require interaction [MDMT23]. Closest to our proposal is a concurrent work by Bontekoe et al. [BAT24]; however, our work differs from theirs in several ways. First, they assume a weaker threat model of non-colluding authorizer and server. Second, their proposed protocols are either interactive or can support only a fixed number of reports, at a higher cost. Finally, our solution outperforms theirs by $100\times$ (Sec. 9). Additionally, some works address the problem in the central model [NFPH15, STC⁺24, BGKW24], assuming a trusted, centralized server and focusing on additive noise mechanisms like Laplace. Other approaches for generating noise in distributed DP settings [DKM⁺06, CasU19, KMS⁺23] also require interaction.

11 Conclusion

We presented two solutions that enable users to generate proofs of honest execution of the randomized response mechanism. At the core of our solutions is a non-interactive verifiable randomness sampling. When inputs are authorized, we use zero-knowledge proofs; otherwise, stronger homomorphic encryption is required.

Acknowledgments. The authors would like to thank Nicholas Genise for their earlier work on this project.

References

- [AJL03] Andris Ambainis, Markus Jakobsson, and Helger Lipmaa. Cryptographic randomized response techniques, 2003.
- [APS15] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015.
- [BAB⁺22] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. OpenFHE: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915, 2022. <https://eprint.iacr.org/2022/915>.
- [BAT24] Tariq Bontekoe, Hassan Jameel Asghar, and Fatih Turkmen. Efficient verifiable differential privacy with input authenticity in the local and shuffle model, 2024.
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334, San Francisco, CA, USA, May 21–23, 2018. IEEE Computer Society Press.
- [BBUV20] Ward Beullens, Tim Beyne, Aleksei Udovenko, and Giuseppe Vito. Cryptanalysis of the legendre PRF and generalizations. *IACR Trans. Symmetric Cryptol.*, 2020(1):313–330, 2020.
- [BC23] Ari Biswas and Graham Cormode. Interactive proofs for differentially private counting. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS ’23*. ACM, November 2023.
- [BF11] Dan Boneh and David Mandell Freeman. Homomorphic signatures for polynomial functions. In Kenneth G. Paterson, editor, *Advances in Cryptology – EUROCRYPT 2011*, pages 149–168, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [BGKW24] Zoë Ruha Bell, Shafi Goldwasser, Michael P. Kim, and Jean-Luc Watson. Certifying private probabilistic mechanisms. Cryptology ePrint Archive, Paper 2024/938, 2024.
- [BGL⁺24] James Bell-Clark, Adrià Gascón, Baiyu Li, Mariana Raykova, and Phillipp Schoppmann. Willow: Secure aggregation with one-shot clients. *IACR Cryptol. ePrint Arch.*, page 936, 2024.
- [BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325, Cambridge, MA, USA, January 8–10, 2012. ACM.
- [BNO08] Amos Beimel, Kobbi Nissim, and Eran Omri. Distributed private data analysis: Simultaneously solving how and what. In *Proceedings of the 28th Annual Conference on Cryptology:*

- Advances in Cryptology*, CRYPTO 2008, pages 451–468, Berlin, Heidelberg, 2008. Springer-Verlag.
- [CasU19] Jeffrey Champion, abhi shelat, and Jonathan Ullman. Securely sampling biased coins with applications to differential privacy. Cryptology ePrint Archive, Paper 2019/823, 2019.
 - [CSS12] T-H. Hubert Chan, Elaine Shi, and Dawn Song. Optimal lower bound for differentially private multi-party aggregation. In *Proceedings of the 20th Annual European Conference on Algorithms*, ESA’12, pages 277–288, Berlin, Heidelberg, 2012. Springer-Verlag.
 - [CSU21] Albert Cheu, Adam Smith, and Jonathan Ullman. Manipulation attacks in local differential privacy. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 883–900. IEEE, 2021.
 - [Dam90] Ivan Bjerre Damgård. On the randomness of legendre and jacobi sequences. In Shafi Goldwasser, editor, *Advances in Cryptology — CRYPTO’ 88*, pages 163–172, New York, NY, 1990. Springer New York.
 - [DJW13] J. C. Duchi, M. I. Jordan, and M. J. Wainwright. Local privacy and statistical minimax rates. In *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, pages 429–438, Oct 2013.
 - [DKM⁺06] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. Our data, ourselves: privacy via distributed noise generation. In *Proceedings of the 24th Annual International Conference on The Theory and Applications of Cryptographic Techniques*, EUROCRYPT’06, page 486–503, Berlin, Heidelberg, 2006. Springer-Verlag.
 - [DR14] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.*, 9:211–407, August 2014.
 - [FGP14] Dario Fiore, Rosario Gennaro, and Valerio Pastro. Efficiently verifiable computation on encrypted data. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’14, page 844–855, New York, NY, USA, 2014. Association for Computing Machinery.
 - [FHS19] Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. *Journal of Cryptology*, 32(2):498–546, April 2019.
 - [GGM84] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *25th FOCS*, pages 464–479, Singer Island, Florida, October 24–26, 1984. IEEE Computer Society Press.
 - [GJNB11] C. C. F. Pereira Geovandro, Marcos A. Simplicio Jr., Michael Naehrig, and Paulo S. L. M. Barreto. A family of implementation-friendly BN elliptic curves. *J. Syst. Softw.*, 84(8):1319–1326, 2011.
 - [GRR⁺16] Lorenzo Grassi, Christian Rechberger, Dragos Rotaru, Peter Scholl, and Nigel P. Smart. MPC-friendly symmetric key primitives. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 430–443, Vienna, Austria, October 24–28, 2016. ACM Press.
 - [KAGN99] H. Kikuchi, J. Akiyama, H. Gobioff, and G. Nakamura. Stochastic voting protocol to protect voters privacy. In *Proceedings 1999 IEEE Workshop on Internet Applications (Cat. No.PR00197)*, pages 103–111, 1999.
 - [KCY21] Fumiyuki Kato, Yang Cao, and Masatoshi Yoshikawa. Preventing manipulation attack in local differential privacy using verifiable randomization mechanism. *CoRR*, abs/2104.06569, 2021.

- [KKK20a] Novak Kaluderovic, Thorsten Kleinjung, and Dusan Kostic. Improved key recovery on the legendre PRF. *IACR Cryptol. ePrint Arch.*, page 98, 2020.
- [KKK20b] Novak Kaluderović, Thorsten Kleinjung, and Dušan Kostić. Cryptanalysis of the generalised legendre pseudorandom function. *ANTS XIV*, 4(1):267–282, 2020.
- [KMS⁺23] Hannah Keller, Helen Möllering, Thomas Schneider, Oleksandr Tkachenko, and Liang Zhao. Secure noise sampling for DP in MPC with finite precision. Cryptology ePrint Archive, Paper 2023/1594, 2023.
- [KVMpAH24] Christian Knabenhans, Alexander Viand, Antonio Merino-Gallardo, and ppp Anwar Hithnawi. vfhe: Verifiable fully homomorphic encryption. In *WAHC@CCS*, pages 11–22. ACM, 2024.
- [MDMT23] Danielle Movsowitz-Davidow, Yacov Manevich, and Eran Toch. Privacy-preserving transactions with verifiable local differential privacy. In *Conference on Advances in Financial Technologies*, 2023.
- [MGSB22] Gonzalo Munilla Garrido, Johannes Sedlmeir, and Matthias Babel. Towards verifiable differentially-private polling. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*, ARES ’22, New York, NY, USA, 2022. Association for Computing Machinery.
- [MN06] Tal Moran and Moni Naor. Polling with physical envelopes: A rigorous analysis of a human-centric protocol. In *Proceedings of the 24th Annual International Conference on The Theory and Applications of Cryptographic Techniques*, EUROCRYPT’06, page 88–108, Berlin, Heidelberg, 2006. Springer-Verlag.
- [NFPH15] Arjun Narayan, Ariel Feldman, Antonis Papadimitriou, and Andreas Haeberlen. Verifiable differential privacy. In *Proceedings of the Tenth European Conference on Computer Systems*, EuroSys ’15, New York, NY, USA, 2015. Association for Computing Machinery.
- [Ode09a] Goldreich Oded. *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge University Press, New York, NY, USA, 1st edition, 2009.
- [Ode09b] Goldreich Oded. *Foundations of Cryptography: Volume 2, Basic Applications*. 2009.
- [rus23] Rust bulletproofs library. <https://github.com/dalek-cryptography/bulletproofs>, 2023. Dalek cryptography Project.
- [San23] Google Privacy Sandbox. Event-level report, attribution reporting api, 2023.
- [SHB23] István András Seres, Máté Horváth, and Péter Burcsi. The legendre pseudorandom function as a multivariate quadratic cryptosystem: security and applications. *Applicable Algebra in Engineering, Communication and Computing*, pages 1–31, 2023.
- [STC⁺24] Ali Shahin Shamsabadi, Gefei Tan, Tudor Ioan Cebere, Aurélien Bellet, Hamed Haddadi, Nicolas Papernot, Xiao Wang, and Adrian Weller. Confidential-DPproof: Confidential proof of differentially private training. In *The Twelfth International Conference on Learning Representations*, 2024.
- [SXZ23] Shaorui Song, Lei Xu, and Liehuang Zhu. Efficient defenses against output poisoning attacks on local differential privacy. *IEEE Transactions on Information Forensics and Security*, 18:5506–5521, 2023.
- [TWM⁺24] Kunal Talwar, Shan Wang, Audra McMillan, Vitaly Feldman, Pansy Bansal, Bailey Basile, Aine Cahill, Yi Sheng Chan, Mike Chatzidakis, Junye Chen, Oliver R. A. Chick, Mona Chitnis, Suman Ganta, Yusuf Goren, Filip Granqvist, Kristine Guo, Frederic Jacobs, Omid Javidbakht, Albert Liu, Richard Low, Dan Mascenik, Steve Myers, David Park, Wonhee Park,

Gianni Parsa, Tommy Pauly, Christian Priebe, Rehan Rishi, Guy N. Rothblum, Congzheng Song, Linmao Song, Karl Tarbe, Sebastian Vogt, Shundong Zhou, Vojta Jina, Michael Scaria, and Luke Winstrom. Samplable anonymous aggregation for private federated data analysis. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 2859–2873, New York, NY, USA, 2024. Association for Computing Machinery.

[vdHLZZ15] Jelle van den Hooff, David Lazar, Matei Zaharia, and Nikolai Zeldovich. Vuvuzela: scalable private messaging resistant to traffic analysis. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, page 137–152, New York, NY, USA, 2015. Association for Computing Machinery.

[War65] Stanley L. Warner. Randomized response: A survey technique for eliminating evasive answer bias. *Journal of the American Statistical Association*, 60(309):63–69, 1965.

A Background Cntd.

Non-Interactive Zero Knowledge Argument of Knowledge. We use non-interactive zero knowledge argument ZK and argument of knowledge ZKPoK for a relation $\mathcal{R}$. A ZK system consists of the following algorithms:

- $\text{crs} \leftarrow \text{ZK.Setup}(1^\lambda, \mathcal{R})$: Generates a common reference string crs for proving the relation $\mathcal{R}$ with security parameter λ .
- $\pi \leftarrow \text{ZK.Prove}(\text{crs}, x, w)$: Generates a proof π that the provers knows the witness w for the statement x such that $(x, w) \in \mathcal{R}$.
- $b \leftarrow \text{ZK.Vf}(\text{crs}, x, \pi)$: Verifies the correctness of proof π for the statement x .

For our purposes, we require the following properties from ZK and ZKPoK.

- Perfect completeness.
- Computational Soundness.
- Computational Zero-Knowledge.

We use two constructions for ZKPoK, namely, (1) Schnorr’s protocols [Ode09a] and (2) Bulletproofs [BBB⁺18].

Blind Signature. Let $\mathcal{U}(\text{pk}, x, \sigma)$ be the composition of user’s algorithms **EQS.SignRequest** and **EQS.Unblind**. For *blindness*, we consider the *malicious signer* model as in the following definition.

Definition 2 (Blindness). *An interactive signing protocol **EQS** satisfies blindness if for all efficient adversary $\mathcal{A}$, the following probability is negligible:*

$$\Pr \left\{ \begin{array}{l} b \leftarrow \$ \{0, 1\}, \\ (\text{pk}, x_0, x_1, \text{st}) \leftarrow \mathbf{EQS.GenParam}(1^\lambda), \\ \text{st}' \leftarrow \mathcal{A}^{\mathcal{U}(\text{pk}, x_b, \cdot)^{(1)}, \mathcal{U}(\text{pk}, x_{1-b}, \cdot)^{(1)}}(\text{st}) \\ \text{Let } \sigma'_b, \sigma'_{1-b} \text{ be the outputs of } \mathcal{U}, \\ \text{set } \sigma'_0 = \sigma'_1 = \perp \text{ if } \sigma'_0 = \perp \text{ or } \sigma'_1 = \perp, \\ b' \leftarrow \mathcal{A}(\text{st}', \sigma'_0, \sigma'_1) \end{array} \right\} b' = b - \frac{1}{2}.$$

B Input Swapping Attack

We illustrate the attack with an example. Consider two malicious users, $\mathcal{U}_1$ and $\mathcal{U}_2$, with inputs $x_1 = 0$ and $x_2 = 1$, who want to report 1s. Suppose the honestly generated noise samples for them are $b_{\rho,1} = 1$

EQS.GenParam(1^κ) - Generate a bilinear group with order p^{BS} of bitlength κ as $\text{BG} = (p^{\text{BS}}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, P, \hat{P}) \leftarrow \text{BGGen}_{\mathcal{R}}(1^\kappa)$. - Set $l = 5$ and generate the key pair for the signature scheme as $(\text{sk}^{\text{BS}}, \text{pk}^{\text{BS}}) \leftarrow \text{KeyGen}_{\mathcal{R}}(1^\kappa)$ where $\text{sk}^{\text{BS}} = (\text{sk}^{\text{BS}}[i])_{i \in [l]} \leftarrow (\mathbb{Z}_p^*)^l, \text{pk}^{\text{BS}} = (\text{pk}^{\text{BS}}[i])_{i \in [l]}$ - Pick $q \leftarrow \mathbb{Z}_{p^{\text{BS}}}^*$ and set $Q \leftarrow qP, \hat{Q} \leftarrow q\hat{P}$. Output. $\text{pprm}_{\mathcal{A}} \leftarrow (\text{BG}, \text{pk}^{\text{BS}}, Q, \hat{Q}), \text{privprm}_{\mathcal{A}} \leftarrow (\text{sk}^{\text{BS}}, q)$
EQS.SignRequest($x, \text{com}_{\text{sk}}, \text{payload}, \text{pprm}_{\mathcal{A}}$) - Parse $\text{pprm}_{\mathcal{A}} = (\text{BG}, \text{pk}^{\text{BS}}, Q, \hat{Q})$ - If $Q = 0_{\mathbb{G}_1}$ or $e(Q, \hat{P}) \neq e(P, \hat{Q})$ then return $\perp$ - Choose $r \leftarrow \mathbb{Z}_p, h \leftarrow \mathbb{Z}_p^*$ and $v \leftarrow \mathbb{Z}_p^*$, and create a commitment to the message x as $C = xP + rQ$ s.t. $C \neq 0_{\mathbb{G}_1}$ - Set $\vec{M} \leftarrow (hC, hP, hvP, hH_{\mathbb{G}_1^*}(\text{com}_{\text{sk}}), hH_{\mathbb{G}_1^*}(\text{payload} \parallel \gamma))$ for $\gamma \leftarrow \{0, 1\}^\kappa$. - Generate the following ZK proofs: $\pi_1 = \text{ZKPoK}\{r \in \mathbb{Z}_p \mid C = xP + rQ\}, \pi_2 = \text{ZK}\{\exists h \in \mathbb{Z}_p^* \mid \vec{M}[2] = hP \text{ and } \vec{M}[1] = hC\}$ Output. $\vec{M}, \gamma, \pi_{\vec{M}} \leftarrow (\pi_1, \pi_2), \text{st} \leftarrow (\text{pprm}_{\mathcal{A}}, \vec{M}, r, h, v, \gamma)$
EQS.VerifyMsg($\vec{M}, x, \pi_{\vec{M}}$) If verification of $\pi_{\vec{M}}$ fails on x, set $b = 0$, otherwise set $b = 1$. Output. b
EQS.Sign($\vec{M}, \text{sk}^{\text{BS}}$) - Parse $\vec{M} = (\vec{M}[i])_{i \in [l]} \in (\mathbb{G}_1^*)^l, \text{sk}^{\text{BS}} = (\text{sk}^{\text{BS}}[i])_{i \in [l]}$ - Choose $d \leftarrow \mathbb{Z}_p^*$ and $c \leftarrow \mathbb{Z}_p^*$ - Compute $\vec{M}[3] = c\vec{M}[3]$ - Compute $Z \leftarrow d(\sum_{i=1}^l \text{sk}_i^{\text{BS}} \vec{M}[i]), D = \frac{1}{d}P, \hat{D} = \frac{1}{d}\hat{P}$ Output. Signature $\sigma \leftarrow (Z, D, \hat{D}), c$
EQS.Verify($\vec{M}, \sigma, \text{pprm}_{\mathcal{A}}$) - Parse $\vec{M} = (\vec{M}[i])_{i \in [l]} \in (\mathbb{G}_1^*)^l, \sigma = (Z, D, \hat{D}), \text{pprm}_{\mathcal{A}} = (\text{BG}, \text{pk}^{\text{BS}}, Q, \hat{Q}), \text{pk}^{\text{BS}} = (\text{pk}_i^{\text{BS}})_{i \in [l]}$ - With $Q \neq 0_{\mathbb{G}_1}$ and $e(Q, \hat{P}) = e(P, \hat{Q})$, set $b = 1$ if the following hold, and 0 otherwise: $\prod_{i=1}^l e(\vec{M}[i], \text{pk}_i^{\text{BS}}) = e(Z, \hat{D}), e(D, \hat{P}) = e(P, \hat{D})$ Output. b
EQS.Unblind(σ, st, c) - Parse $\sigma = (Z, D, \hat{D}), \text{st} = (\text{pprm}_{\mathcal{A}}, \vec{M}, r, h, v, \gamma), \text{pprm}_{\mathcal{A}} = (\text{BG}, \text{pk}^{\text{BS}}, Q, \hat{Q}), \vec{M} = (\vec{M}[i])_{i \in [l]} \in (\mathbb{G}_1^*)^l$ - Set $\vec{M}^{\dagger} = (\vec{M}[1], \vec{M}[2], c\vec{M}[3], \vec{M}[4], \vec{M}[5])$ - If EQS.Verify($\vec{M}^{\dagger}, \sigma, \text{pprm}_{\mathcal{A}}$) = 0, return $\perp$ - Choose $\psi \leftarrow \mathbb{Z}_p^*$ and compute $\sigma' \leftarrow (\psi \frac{1}{h} Z, \frac{1}{\psi} D, \frac{1}{\psi} \hat{D})$ Output. $\vec{M}' = \frac{1}{h} \vec{M}^{\dagger}, \sigma'$
EQS.VerifyToken($\sigma', \vec{M}', \text{pprm}_{\mathcal{A}}, \text{com}_{\text{sk}}, \text{payload}, \gamma$) - Parse $\vec{M}' = (\vec{M}'[i])_{i \in [l]} \in (\mathbb{G}_1^*)^l$ - If $(\vec{M}'[2] \neq P \text{ OR } \vec{M}'[4] \neq H_{\mathbb{G}_1^*}(\text{com}_{\text{sk}}) \text{ OR } \vec{M}'[5] \neq H_{\mathbb{G}_1^*}(\text{payload} \parallel \gamma))$, set $b = 0$ - Set $b \leftarrow \text{EQS.Verify}(\vec{M}', \sigma', \text{pprm}_{\mathcal{A}})$ Output. b

Figure 7: EQS Blind Signature. Steps marked in blue indicate the modifications done to the standard signature as discussed in Sec. 4.2.

Parameters. ϵ - Privacy parameter; κ - Security parameter

• **Setup Phase.**

Server and User:

- Both the server $\mathcal{S}$ and user $\mathcal{U}_i$ jointly generates the parameters of the threshold HE scheme $(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{U}}^{\text{HE}}, \text{sk}_{i,\mathcal{S}}^{\text{HE}}) \leftarrow \text{HE.GenParam}(1^\kappa, 2, 2)$, where $\text{sk}_{i,\mathcal{S}}^{\text{HE}}$ and $\text{sk}_{i,\mathcal{U}}^{\text{HE}}$ is known only to the server and the user $\mathcal{U}_i$, respectively.

User: Each user $\mathcal{U}_i$:

- Verifies the proof $\pi_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}}$ and aborts in case of the check fails
- Samples its share of the secret key for the PRF $\text{sk}_{i,\mathcal{U}}^{\mathcal{F}} \leftarrow \$_{\mathbb{F}_p}$
- Encrypts the sampled key and generates a ZK proof of the correctness of the encryption

$$\text{ct}_{\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}} \leftarrow \text{HE.Enc}(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}}), \pi_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}} = \text{ZKPoK}\{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}} \in \mathbb{F}_p \mid \text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}} = \text{HE.Enc}(\text{pk}_i^{\text{HE}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}})\}$$

- Sends the generated ciphertext and ZK proof to the server $\mathcal{U}_i \xrightarrow{\text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \pi_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}} \mathcal{S}$

Server: For each user $\mathcal{U}_i$:

- Continue if $\pi_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}$ verifies, reject otherwise.

• **Input Authorization Phase.**

User: Each user $\mathcal{U}_i$

- Prepares the message for the signature based on its input bit x_i : $(\vec{M}_i, \pi_{\vec{M}_i}, \text{st}) \leftarrow \text{EQS.SignRequest}(x_i, \text{pprm}_{\mathcal{A}})$
- Stores the state information st with itself
- Sends the message $\vec{M}_i$ and the associated proof $\pi_{\vec{M}_i}$ to the authorizer $\mathcal{U}_i \xrightarrow{\vec{M}_i, \pi_{\vec{M}_i}} \mathcal{A}$

Authorizer:

- Authorizer collects the messages and proofs for each user, and issues a blind signature on it after verifying the proof

$$b \leftarrow \text{EQS.VerifyMsg}(\vec{M}_i, \pi_{\vec{M}_i})$$

- In case the proof is valid, the authorizer issues a blind signature on the message and sends it to the user

$$\sigma_i \leftarrow \text{EQS.Sign}(\vec{M}_i, \text{sk}_{\mathcal{A}}), \mathcal{A} \xrightarrow{\sigma_i} \mathcal{U}_i$$

User:

- Unblinds the signature to obtain a new signature and token τ_i

$$(\vec{M}'_i, \sigma'_i) \leftarrow \text{EQS.Unblind}(\sigma_i, \text{st}_i, \text{pprm}_{\mathcal{A}}), \tau_i = H_{\mathbb{F}_p}(\vec{M}'_i[3])$$

• **Noise Sampling Phase.**

User: Each user $\mathcal{U}_i$

- Encrypt the input bit x_i and generate a ZK proof of the correctness of the encryption

$$\text{ct}_{x_i} \leftarrow \text{HE.Enc}(\text{pk}_i^{\text{HE}}, x_i); \pi_{x_i} = \text{ZKPoK}\{z \in \{0, 1\} \mid z = \text{Open}(\vec{M}'_i[1]) \text{ and } \text{ct}_{x_i} = \text{HE.Enc}(\text{pk}_i^{\text{HE}}, z)\}$$

- Sets $k \leftarrow \lfloor \log_2(1 + e^\epsilon) \rfloor$ and $f_{\tau_i}(\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}, x_i) = x_i + (1 - 2x_i) \prod_{j=1}^k \mathcal{F}(\text{sk}_{i,\mathcal{U}}^{\mathcal{F}} + \tau_i, j)$
- Homomorphically evaluate f_{τ_i} to generate the ciphertext of the response $\text{ct}_{y_i} \leftarrow \text{HE.Eval}(\text{pk}_i^{\text{HE}}, \text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \text{ct}_{x_i}, f_{\tau_i})$
- Partially decrypt ct_{y_i} using the secret key $\text{sk}_{i,\mathcal{U}}^{\text{HE}}$ and generate a ZK proof showing the correctness of this step

$$d_{i,\mathcal{U}} = \text{HE.PartialDec}(\text{sk}_{i,\mathcal{U}}^{\text{HE}}, \text{ct}_{y_i}), \pi_{y_i} = \text{ZKPoK}\{\text{sk}_{i,\mathcal{U}}^{\text{HE}} \mid d_{i,\mathcal{U}} = \text{HE.PartialDec}(\text{sk}_{i,\mathcal{U}}^{\text{HE}}, \text{ct}_{y_i})\}$$

- Send all the computed ciphertexts, signature σ' , message $\vec{M}'_i$, the partial decryption of the response and the ZK proofs to the server

$$\mathcal{U}_i \xrightarrow{\text{ct}_{y_i}, \text{ct}_{x_i}, \sigma'_i, \vec{M}'_i, d_{i,\mathcal{U}}, \pi_{x_i}, \pi_{y_i}} \mathcal{S}$$

• **Verification of Computation Phase.**

Server: For each user $\mathcal{U}_i$, the server computes the following:

- Verify the signature and message pair $(\sigma'_i, \vec{M}'_i)$, and reject if invalid.
- Verify the proof π_{x_i} on ct_{x_i} and reject in case of an invalid proof.
- Re-compute the ciphertext for the response $\text{ct}'_{y_i} = \text{HE.Eval}(\text{pk}_i^{\text{HE}}, \text{ct}_{\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}}, \text{ct}_{x_i}, f_{\tau_i})$ where $\tau_i = H_{\mathbb{F}_p}(\vec{M}'_i[3])$
- Continue only if $\text{ct}_{y_i} = \text{ct}'_{y_i}$ and π_{y_i} verifies. Reject otherwise.
- Generate its decryption share for ct_{y_i} and combine $(d_{i,\mathcal{U}}, d_{i,\mathcal{S}})$ for full decryption.

$$d_{i,\mathcal{S}} \leftarrow \text{HE.PartialDec}(\text{sk}_{i,\mathcal{S}}^{\text{HE}}, \text{ct}_{y_i}); y_i \leftarrow \text{HE.Combine}(\text{pk}_i^{\text{HE}}, d_{i,\mathcal{U}}, d_{i,\mathcal{S}})$$

Figure 8: *Verity-Auth*⁺: Verifiable randomized response where the users have authorized input and the randomness is kept encrypted.

and $b_{\rho,2} = 0$, which forces both to report 0s. However, if they swap their inputs, they can both report 1s. Specifically, they can do this by swapping their message and signature pairs when submitting to the server, which is possible due to the unlinkability of the signatures. Note that this attack is randomness-dependent, which should be disallowed by our protocol. By binding the secret key to the message, we prevent such an attack, as the user’s identity is tied to the input they report *before* observing the randomness.

C Multiple Messages

Verity-Auth provides verifiability for the input obtained from the authorizer while allowing the user to select any corresponding payload. We can think of this as bounding the ℓ_1 norm of a user’s contribution to a histogram defined over the payload domain. This functionality can be used to enforce that users can contribute value to a single demographic bucket in the medical test reporting application, or it can guarantee that a user cannot report more conversions (i.e., engaging with an ad converting into a purchase) than they have in reality in the private advertising application.

In some cases, further restrictions on allowable payload values may be needed. To enforce this, we can extend the user’s request for a blind signature with a ZK proof that the payload and input satisfy these additional constraints. This could, for example, verify that the correct attribution logic has been applied in private conversion measurement.

D Benchmark Results of Bontekoe et al. [BAT24]

As a comparison, we also benchmarked the VLDP protocol of Bontekoe et al. [BAT24]. Specifically we benchmarked the Expand and the Shuffle model variants. Note that the VLDP protocols in [BAT24] are based on an k -ary RR that outputs either the input x or a random k -ary value based on a Bernoulli bit, and their schemes approximately sample the Bernoulli bit by comparing a uniformly random bitstring of length ℓ with a threshold. As a result, the performance is independent of both the privacy parameter ϵ and the number of users, and it is mainly determined by the length ℓ of the random source to guarantee that the approximate samples are sufficiently close to the true distribution. For a fair comparison, we set $k = 2$ in their implementation, and kept their approximate sampling intact.

The VLDP protocol implementation uses the Blake2e block cipher as a PRF to generate the randomness for approximate sampling, and it uses Groth16 to let the user prove its PRF evaluation. The Expand variant of VLDP protocol allows to generate the randomness needed by 2^{d-1} RR invocations with a single interactive round between the user and the server, by letting the user commit to its randomness in a Merkle tree of depth d . In our benchmark we set $d = 4$. It takes the server 729.97ms to generate the parameters used by the protocol. The 3-round interactive randomness generation phase runs in 7.97ms, where the user’s first round runs in 5.05ms, the server’s runs in 2.67ms, and the user’s third round runs in 0.23ms. To report a single verifiable LDP contribution, the user program runs in 818.79ms and the server program runs in 3.79ms, where majority of both the user time (808.57ms) and the server time (2.86) is spent on generating and verifying, respectively, the Groth16 ZK proof of correct PRF evaluation.

The Shuffle model variant hides the user’s identity by including identifiable key materials as witnesses of ZK proofs. As a result, parameter generation takes 1.23s as the ZK proofs require more constraints. The interactive randomness generation protocol runs in 2.7ms in total. To report a single verifiable LDP contribution, it takes each user 1.29s to generate the Groth16 ZK proof of PRF evaluation, and the whole user program runs in 1.31s; the server program on the other hand runs in 3.09ms where Groth 16 verification takes 2.59ms.

E Security Analysis Cntd.

We analyze the security of our two constructions in the real-ideal world security model [Ode09b] assuming malicious adversary who can make the parties it controls deviate arbitrarily from the protocol. We describe the ideal privacy properties for the output of our protocols in an ideal world assuming a trusted functionality, which obtains the inputs of all honest parties and instructions from the adversary indicative of the misbehavior it can perform, and then outputs the prescribed output. The real world is the actual execution of the protocol. The security argument shows that an adversary cannot distinguish the real and the ideal executions, which implies that the real execution cannot reveal more than the output from the ideal world.

F Proof of Theorem 4

Proof. Honest Server case: Let us start by considering the case with only malicious users. The simulator proceeds as follows.

First the simulator creates a simulated authorizer and server to interact with the adversarial users. It then runs through the protocol with the malicious users. From the ZKPoK that the malicious users send in during setup the simulator extracts their secret keys. When the zero-knowledge proofs are sent in it extracts the secrets from the ZK proofs including the implicit x_i^A and h .

The simulator now uses h and the fact that any hashed value must have been submitted to the random oracle that the simulator is running (and thus be known to the simulator). This allows it to derive the committed com_{sk_i} and payload_i . These steps might fail if the user isn't hashing anything to make their message, but this will not be a problem. Then the simulator submits each x_i as input to the ideal functionality along with the corresponding payload (or with an arbitrary payload if no payload could be derived).

Let $\mathcal{I}_A$ be the set of users the simulator receives a y_i for from the ideal functionality. This is the set of users for which the honest Authorizer had a matching x_i^A , for the other inputs the honest Authorizer must have had the opposite bit. The simulated Authorizer thus ignores the malicious user messages that either used the wrong x_i or were badly formed and only replies to the correct ones, as follows.

For each client with a valid submission the simulated authorizer will reply with a blind signature. If the simulator was unable to identify the preregistered com_{sk} and thus sk for that submission it honestly generates a signature. However, if it knows the sk it will repeatedly generate signatures with new randomness until it produces one which results in the correct y_i i.e. the y_i that the user (behaving honestly) would calculate from the provided signature would match that from the ideal functionality. Either way the resulting signature is then sent to the simulated malicious user.

The simulator then receives the messages from the users to the server. The simulator simulates a server processing, i.e. verifying, them all. It then tells the ideal functionality that the set of non-dropout (corrupt) users is exactly the set who submitted valid messages for the server to verify.

Honest Server Hybrids: Let $\mathcal{H}^0$ be the view of the adversary and output of the server in the real world. Let $\mathcal{H}^1$ be the output with the zero knowledge proof extractions performed. This is indistinguishable by the security of the zero-knowledge proof system.

Let $\mathcal{H}^2$ be $\mathcal{H}^1$ but with the simulated authorizer instead of the real one and the real server altered to think the verification key generated by the simulated authorizer is the verification key of the real authorizer. We now show $\mathcal{H}^1$ is indistinguishable from $\mathcal{H}^2$. Firstly, note that the set of users receiving a reply is exactly the same in both cases as the simulated authorizer verifies proofs the same way and is told by the ideal functionality who the real authorizer would drop for bad x_i . Secondly, the distribution of the y_i (conditional on the adversary's previous view) is indistinguishable in the two cases due to the pseudorandomness of the Legendre PRF, the fact its output is used to compute the same probability the ideal functionality is using and that the input τ_i is unpredictable by the adversary in advance. That unpredictability is a consequence of the security of the EQS, specifically that the τ_i could not have been forged in advance.

Let $\mathcal{H}^3$ be the view in the ideal world and ideal server output. $\mathcal{H}^3$ is with high probability equal to $\mathcal{H}^2$. This is guaranteed by the security of the final zero-knowledge proofs, the binding of the secret key commitments and the collision resistance of the random oracle. Cumulatively these guarantee that there is a negligible chance of the adversary submitting a final message which doesn't result in the output contribution that the simulator already sent to the ideal functionality.

Corrupt Server case: We now consider the corrupt server case. This case is easier in the sense that we now have no honest outputs to assure the correctness of, however we need to ascertain that the malicious server can't learn too much.

Our simulator will first honestly generate and register secret keys for the honest clients with the server i.e. adversary. It then receives the messages from corrupt users for the authorizer. It extracts the x_i^A and the hash of payload_i from the message by extracting h from the ZKPoK and then uses the fact that it sees all random oracle queries to identify the payload_i (of all legitimately constructed messages). It then provides $(x_i, \text{payload}_i)$ to the ideal functionality as the input of user i . If it receives y_i (for corrupt i) then it knows that the x_i was provided honestly and signs request i on behalf of a simulated authorizer and sends it to the adversarial user, otherwise it drops that user. It then replies to the ideal functionality saying all the malicious users have chosen to drop out.

Next it receives from the ideal functionality the honest users contributions $(y_i, \text{payload}_i)$. For each such contribution it receives it simulates an honest interaction between a user (with the secret keys committed to earlier) and the simulated authorizer resulting in that contribution i.e. uses rejection sampling to find authorizer randomness that gives the correct y_i . Each resulting message is sent to the server i.e. adversary. The adversary's output is then returned by the simulator.

Corrupt Server Hybrids:

Let $\mathcal{H}^0$ be the real world view of the adversary. Let $\mathcal{H}^1$ be the output with the zero knowledge proof extractions performed. This is indistinguishable by the security of the zero-knowledge proof system.

Let $\mathcal{H}^2$ be like $\mathcal{H}^1$ except with the authorizations from the simulated authorizer rather than the real authorizer (and the public key for the authorizer known to the server matching the simulated authorizer's key). These hybrids are identically distributed because the simulated authorizer runs exactly the same code as the real one except in choosing to drop clients for incorrect x_i . However it still drops exactly the clients with incorrect x_i because it is told exactly who they are by the ideal functionality dropping them.

Let $\mathcal{H}^3$ be as $\mathcal{H}^2$ except with the real honest user messages to the server replaced by simulated ones i.e. the view in the ideal world. The payloads will match and the y_i will have the same distribution meanwhile the blind signatures blindness properties and the ZK proofs zero-knowledge properties guarantee that the other components of the message leak nothing about the underlying x_i and committed secret keys. Thus these hybrids are indistinguishable. $\square$

G Proof of Theorem 5

Proof. We prove here only the case with the shuffling included as it is the more complicated case.

Corrupt Authorizer Simulator: For this case we must make use of the fact that the simulator can have a copy of the authorizer's true input i.e. the inputs of the honest users. The simulator proceeds as follows.

First it has the users send in honestly generated commitments com_{sk_i} for randomly generated secret keys together with the ZKPoKs. It can then forget the keys as it will not need them later.

Next the simulator simulates the honest users, with the correct x_i (that it has access to because the corrupt authorizer was also given them) and uniform randomness r_i in place of $H_{\mathbb{G}_1^*}(\text{payload}_i || \gamma_i)$, to make submissions to the adversarial authorizer. For each honest user i that the authorizer provides a valid blind signature σ_i to the simulator sends the true input x_i to the ideal functionality. For each user i the authorizer doesn't

provide a valid signature to the simulator sends $1 - x_i$. The simulator makes no submissions to the ideal functionality on behalf of corrupt users, effectively dropping them from the ideal functionality. It then receives the contributions $(y, \text{payload})$ for the honest parties.

Each $(y, \text{payload})$ returned is paired up with a (different valid) σ_i . The simulator now generates a random γ_i and reprograms the random oracle so that $H_{G_1^*}(\text{payload} \parallel \gamma_i) = r_i$. To wrap up as the sk_i may not actually result in y_i from x_i , the simulator fakes the zero knowledge proof and submits it to the server along with σ_i, y and payload . It then outputs whatever the adversary outputs.

Corrupt Authorizer Hybrids: Let $\mathcal{H}^0$ be the adversary's view and honest user's inputs in the real world. Let $\mathcal{H}^1$ be same except with the random oracle reprogrammed so that $H_{G_1^*}(\text{payload}_i \parallel \gamma_i) = r_i$. This change doesn't change the distribution of the view as the random oracle is still random.

Let $\mathcal{H}^2$ be as $\mathcal{H}^1$ except the y_i with each payload_i is changed to match what the ideal functionality chose that y_i to be in the ideal world, and the proofs that they are produced correctly are faked. The distribution of the y_i is indistinguishable as the user's secret key was high entropy to the adversary, the Legendre PRF is pseudorandom and the protocol itself computed the y_i to have the same distribution as the ideal world. Meanwhile the faked proofs are indistinguishable by virtue of the fake proof property.

Let $\mathcal{H}^3$ be as $\mathcal{H}^2$ except with the pairing of σ_i and $(y, \text{payload})$ as in the ideal world, the corresponding fake proofs updated and the random oracle yet again reprogrammed so that the $H_{G_1^*}(\text{payload} \parallel \gamma_i) = r_i$ for the i the payload are now paired with. The reprogramming is again undetectable because of the entropy of the γ_i . The proof tampering is undetectable by the fake proof property. The switching of the σ_i themselves is undetectable due to the fact that EQS is a blind signature scheme meaning that the authorizer (not the server) can tell which σ_i was which.

$\mathcal{H}^3$ is the view for the ideal world so we are done. □

H Proof of Theorem 6

Proof. Honest Server Simulator: If the server is honest then the simulators interaction with the malicious users in this protocol is straightforward. It just simulates a server interacting honestly with the malicious users with the sole exception that it extracts x_i from π_{x_i} . It then sends the x_i to the ideal functionality and outputs whatever the adversary outputs.

Honest Server Hybrids: We ignore the influence of honest users as the computation with them happens completely independently to that with the corrupt users.

Let $\mathcal{H}^0$ be the real world view of the adversary and the servers output. Let $\mathcal{H}^1$ be the same but with the server's random tape matching that used by the simulator for simulating the server. This clearly has an identical distribution.

Let $\mathcal{H}^2$ be the result of replacing the output in $\mathcal{H}^1$ with the ideal world output i.e. the ideal world view and output. The secret key of this server is pseudorandom to the adversary by the security of the HE scheme, the security of the ZKPoK from the server and the determinacy of the server's ZKPoK verification. Thus the adversary's view is computationally independent of the output in $\mathcal{H}^1$, as said output is independent of the clients input if the server's secret key is uniformly random. Thus it is sufficient to prove that the distributions of the output are the same. This is immediate from the pseudorandomness of the secret key, the pseudorandomness of the Legendre PRF and the calculation in the protocol producing the right distribution if the random bits from the PRF were true randomness.

Corrupt Server Simulator: The ideal functionality will provide no output to honest parties and the corrupt parties input has no effect on the information returned about the honest parties input. Therefore the simulator can start by providing arbitrary inputs for the corrupt users and then receiving all the outputs from

the ideal functionality. The simulator now needs to simulate honest users interacting with the adversarial protocol so that the server ends up receiving the y_i that the ideal functionality provided.

To achieve this the simulator simulates honest users interacting with the adversary. They behave honestly throughout except for two things. Firstly, when the server provides ZKPoKs the simulator extracts the witness. Secondly, it uses the knowledge of $\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}$ (and $\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}$) to compute and set $x_i = f_{\text{cnt}_i}(\text{sk}_{i,\mathcal{S}}^{\mathcal{F}}, \text{sk}_{i,\mathcal{U}}^{\mathcal{F}}, y_i)$ i.e. what x_i needs to be set to to get the y_i to come out right. Finally the simulator outputs whatever the adversary does.

Corrupt Server Hybrids: Let $\mathcal{H}^0$ be the real world view of the adversary. Let $\mathcal{H}^1$ be the same but with the true input replaced by x'_i , i.e. the ideal world. Note that due to the adversary having no knowledge of $\text{sk}_{i,\mathcal{U}}^{\mathcal{F}}$ the y_i in $\mathcal{H}^0$ has the correct distribution (as in the proof that $\mathcal{H}^1 = \mathcal{H}^2$ in the honest server proof). Indistinguishability follows from the security of the HE scheme and the fact that the ZKPoK is ZK. $\square$

I Proof of Lemma 7

Proof. In case of no drop outs, the ideal functionality of *Verity-Auth* is given by Fig. 9 where *all* users submit valid inputs to the functionality. Assuming $\mathcal{I} = n$, the proof then follows directly from it. $\square$

J Proof of Lemma 8

Proof. Let $w = \sum_{j=1}^m y_i$. From the ideal functionality of Fig. 5, we see that the $\mathcal{U}_i$ s can drop out on the basis of y_i . In the worst case, all the malicious users might drop out, which means that the observed poisoned aggregate by the server is given by $\hat{s}' = \sum_{i=1}^{|\mathcal{I}|} y_i - w$. Assuming $\mathcal{I} = n$, it is easy to see that,

$$\begin{aligned} \Pr\left[\sum_{j=1}^m y_i = w, w \in \{0, m\}\right] &= \begin{cases} \sum_{j=0}^w \left(\binom{q}{j} (1-\rho)^j \rho^{q-j} \binom{m-q}{w-j} \rho^{w-j} (1-\rho)^{m+j-q-w} \right) \\ \quad \text{if } q \leq \frac{m}{2} \text{ and } w \leq q \\ \sum_{j=0}^q \left(\binom{q}{j} (1-\rho)^j \rho^{q-j} \binom{m-q}{w-j} \rho^{w-j} (1-\rho)^{m+j-q-w} \right) \\ \quad \text{if } q \leq \frac{m}{2} \text{ and } q < w \leq m - q \\ \sum_{j=w+q-m}^q \left(\binom{q}{j} (1-\rho)^j \rho^{q-j} \binom{m-q}{w-j} \rho^{w-j} (1-\rho)^{m+j-q-w} \right) \\ \quad \text{if } q \leq \frac{m}{2} \text{ and } m - q < w \leq m \\ \sum_{j=0}^w \left(\binom{q}{j} (1-\rho)^j \rho^{q-j} \binom{m-q}{w-j} \rho^{w-j} (1-\rho)^{m+j-q-w} \right) \\ \quad \text{if } q > \frac{m}{2} \text{ and } w \leq m - q \\ \sum_{j=w+q-m}^w \left(\binom{q}{j} (1-\rho)^j \rho^{q-j} \binom{m-q}{w-j} \rho^{w-j} (1-\rho)^{m+j-q-w} \right) \\ \quad \text{if } q > \frac{m}{2} \text{ and } m - q < w \leq q \\ \sum_{j=w+q-m}^q \left(\binom{q}{j} (1-\rho)^j \rho^{q-j} \binom{m-q}{w-j} \rho^{w-j} (1-\rho)^{m+j-q-w} \right) \\ \quad \text{if } q > \frac{m}{2} \text{ and } q < w \leq m \end{cases} \\ &= \sum_{j=\max\{0, w+q-m\}}^{\min\{q, w\}} \left(\binom{q}{j} (1-\rho)^j \rho^{q-j} \binom{m-q}{w-j} \rho^{w-j} (1-\rho)^{m+j-q-w} \right) \\ &= \sum_{j=\max\{0, w+q-m\}}^{\min\{q, w\}} \left(\binom{q}{j} \binom{m-q}{w-j} (1-\rho)^{m+2j-q-w} \rho^{q+w-2j} \right) \end{aligned}$$

The proof then follows from the above equation and Thm. V.7 from [CSU21]. $\square$

Parameters. ϵ - Privacy parameter User's Input. Each user $\mathcal{U}_i$ inputs a bit x_i and chooses payload_i Authorizer's Input. Authorizer inputs the ground truth x_i^A for every user $\mathcal{U}_i$ Functionality. • Set $k \leftarrow \lfloor \log_2(1 + e^\epsilon) \rfloor$ and $\rho = \frac{1}{2^k}$ • Let $\mathcal{I}$ be the set of users whose input x_i matches the x_i^A from the authorizer $\mathcal{A}$ • For every user $\mathcal{U}_i \in \mathcal{I}$ – Generate a sample of $b_{i,\rho} \sim \text{Ber}(\rho)$ – Set $y_i = (1 - x_i^A)b_{i,\rho} + (1 - b_{i,\rho})x_i^A$ • Send $(y_i, \text{payload}_i)_i$ to the server $\mathcal{S}$
--

Figure 9: Ideal Functionality for Verity-Auth^+

K Proof of Lemma 9

Proof. The ideal functionality of Verity-Auth^+ is given by Fig. 9 where a user who chooses to drop out (based on their input) provides an invalid/inconsistent input. In the worst-case, all the malicious users might choose to drop out. So this means, now the true, non-privatized aggregate is given by $s' = \sum_{i=1}^{|\{\mathcal{U}_i\}|} x_i - q$ where $n = |\{\mathcal{U}_i\}|$ which completes our proof. In particular q is the bias introduced in the non-privatized aggregate itself, while the $\frac{\sqrt{n}}{\epsilon}$ term corresponds to the error due to the noise. $\square$

L Proof of Lemma 10

Proof. From the ideal functionality of Verity^i given by Fig. 6, we can see that the users fully control their input and are therefore able to carry out input poisoning attacks. The proof then follows directly Lemma 3 where $n = |\{\mathcal{U}_i\}|$. Note that in this case drop outs provide no extra advantage to the adversary. $\square$